

Notifications APIs Developer Guide

Release 1.0.0-beta.2

NVIDIA Corporation

Oct 20, 2026

Contents

1 What is the Notifications Service? 2

2 Architecture Overview 3

3 Key Concepts 4
3.1 Events . . . 4
3.2 Event Types and Message Contracts . . . 4
3.3 Durable vs Non-Durable Queues . . . 5
3.4 Resource Filtering . . . 5

4 Available APIs 6

5 Documentation Structure 7
5.1 Getting Started . . . 7
5.1.1 Prerequisites . . . 7
5.1.2 Authentication . . . 7
5.1.3 Quick Example: Publish and Consume an Event . . . 8
5.1.3.1 Step 1: Publish an Event (REST) . . . 8
5.1.3.2 Step 2: Consume Events (REST with SSE) . . . 9
5.1.4 Quick Example: gRPC . . . 9
5.1.4.1 Step 1: Generate Client Code . . . 9
5.1.4.2 Step 2: Publish an Event (gRPC) . . . 9
5.1.4.3 Step 3: Consume Events (gRPC) . . . 10
5.1.5 Common Patterns . . . 11
5.1.5.1 1. Define Your Event Types . . . 11
5.1.5.2 2. Define Your Message Schemas . . . 11
5.1.5.3 3. Handle Errors . . . 12
5.1.6 Next Steps . . . 12
5.1.7 Troubleshooting . . . 12
5.1.7.1 “Permission denied” errors . . . 12
5.1.7.2 “Service unavailable” errors . . . 12
5.1.7.3 Events not being received . . . 13
5.1.7.4 Connection drops . . . 13
5.2 Publishing Events . . . 13
5.2.1 Overview . . . 13
5.2.2 Event Structure . . . 13
5.2.2.1 Event Type . . . 14
5.2.2.2 Message . . . 14
5.2.2.3 Occurred At . . . 15
5.2.2.4 Resource (Optional) . . . 15
5.2.3 Publishing a Single Event . . . 15
5.2.3.1 REST API . . . 15
5.2.3.2 gRPC API . . . 16
5.2.4 Batch Publishing Multiple Events . . . 19

5.2.4.1 REST API . . . 19
5.2.4.2 gRPC API . . . 20
5.2.5 Error Handling . . . 21
5.2.5.1 Common Error Codes . . . 21
5.2.5.2 Retry Logic . . . 21
5.2.6 Best Practices . . . 22
5.2.6.1 1. Choose Appropriate Event Granularity . . . 22
5.2.6.2 2. Include Timestamps Accurately . . . 23
5.2.6.3 3. Use Resource IDs Consistently . . . 23
5.2.6.4 4. Batch When Possible . . . 23
5.2.6.5 5. Handle Partial Batch Failures . . . 24
5.2.6.6 6. Keep Messages Reasonably Sized . . . 24
5.2.6.7 7. Document Your Events . . . 24
5.2.7 Performance Considerations . . . 25
5.2.8 Next Steps . . . 25
5.3 Consuming Events . . . 25
5.3.1 Overview . . . 25
5.3.2 Durable vs Non-Durable Queues . . . 25
5.3.2.1 Non-Durable Queues . . . 25
5.3.2.2 Durable Queues . . . 26
5.3.2.3 Decision Matrix . . . 27
5.3.2.4 Key Principle . . . 28
5.3.3 Consuming with Non-Durable Queues . . . 28
5.3.3.1 REST API . . . 28
5.3.3.2 gRPC API . . . 33
5.3.4 Consuming with Durable Queues . . . 38
5.3.4.1 Step 1: Create a Durable Queue . . . 38
5.3.4.2 Step 2: Consume from Durable Queue . . . 41
5.3.4.3 Durable Queue Lifetime . . . 44
5.3.4.4 Bounded Redelivery and the Dead-Letter Queue . . . 45
5.3.4.5 Managing the Dead-Letter Queue (Operators) . . . 46
5.3.4.6 Step 3: Delete Durable Queue . . . 50
5.3.4.7 Updating Durable Queue Filters . . . 51
5.3.5 Resource Filtering . . . 52
5.3.5.1 Filter Types . . . 52
5.3.5.2 Empty Resource ID . . . 53
5.3.5.3 Multiple Filters (OR Logic) . . . 53
5.3.5.4 Multiple Event Types . . . 53
5.3.6 Error Handling . . . 54
5.3.6.1 Authorization Errors During Streaming . . . 54
5.3.6.2 Other Common Errors . . . 56
5.3.7 Best Practices . . . 57
5.3.7.1 1. Choose the Right Queue Type . . . 57
5.3.7.2 2. Handle Disconnections Gracefully . . . 57
5.3.7.3 3. Process Events Idempotently . . . 59
5.3.7.4 4. Don’t Block the Event Stream . . . 59
5.3.7.5 5. Monitor Queue Depth (Durable Queues) . . . 60
5.3.7.6 6. Delete Durable Queues When Done . . . 60
5.3.7.7 7. Use Appropriate Filters . . . 60
5.3.8 Complete Example: Thumbnail Generation Service . . . 61
5.3.9 Next Steps . . . 63
5.4 Permissions & Authorization . . . 64
5.4.1 Overview . . . 64
5.4.2 Authentication . . . 64

5.4.2.1 Including Auth Token . . . 64
5.4.2.2 Token Expiration . . . 64
5.4.3 Publishing Permissions . . . 65
5.4.3.1 Event Type Permissions . . . 65
5.4.3.2 Example Policy . . . 65
5.4.3.3 REST Example . . . 65
5.4.3.4 Configuration . . . 66
5.4.4 Consuming Permissions . . . 66
5.4.4.1 Durable Queue Permissions . . . 66
5.4.4.2 Durable Queue Event Permissions . . . 67
5.4.4.3 Non-Durable Queue Permissions . . . 68
5.4.5 Storage Events: Special “Docs” Permissions . . . 68
5.4.5.1 Storage Event Types . . . 68
5.4.5.2 How Storage Permissions Work . . . 68
5.4.5.3 Why “Docs” Permissions? . . . 69
5.4.5.4 Configuration for Storage Events . . . 70
5.4.5.5 Storage Permissions with Durable Queues . . . 70
5.4.6 Fine-Grained Permissions for Arbitrary Event Types . . . 71
5.4.6.1 When to Use Fine-Grained Permissions . . . 71
5.4.6.2 Configuration . . . 71
5.4.6.3 How It Works . . . 71
5.4.6.4 Permissions API Call . . . 72
5.4.6.5 Policy Examples . . . 72
5.4.6.6 Complete Configuration Example . . . 73
5.4.7 Permissions Decision Flow . . . 74
5.4.7.1 Non-Durable Queues . . . 74
5.4.7.2 Durable Queues . . . 75
5.4.8 Summary Table . . . 75
5.4.9 Configuration Examples . . . 76
5.4.9.1 Example 1: Thumbnail Service (Durable Queue) . . . 76
5.4.9.2 Example 2: User Dashboard (Non-Durable Queue) . . . 76
5.4.9.3 Example 3: Project Notification Service (Non-Durable + Fine-Grained) . . . 76
5.4.10 Best Practices . . . 77
5.4.10.1 1. Use Service Accounts for Durable Queues . . . 77
5.4.10.2 2. Document Your Permission Requirements . . . 77
5.4.10.3 3. Test Permissions Thoroughly . . . 78
5.4.10.4 4. Monitor Permission Denials . . . 78
5.4.10.5 5. Use Hierarchical Policies Where Appropriate . . . 78
5.4.11 Troubleshooting . . . 78
5.4.11.1 “Permission denied” when publishing . . . 78
5.4.11.2 “Permission denied” when creating durable queue . . . 79
5.4.11.3 Not receiving storage events (non-durable) . . . 79
5.4.11.4 Not receiving events with fine-grained permissions . . . 79
5.4.11.5 Token expiring during stream . . . 79
5.4.12 Next Steps . . . 79
5.5 API Reference . . . 79
5.5.1 API Types . . . 80
5.5.2 Event Aggregation Service (Publisher API) . . . 80
5.5.2.1 Base URLs . . . 80
5.5.2.2 Authentication . . . 80
5.5.3 Publisher gRPC API . . . 80
5.5.3.1 Service Definition . . . 80
5.5.3.2 PublishEvent . . . 81
5.5.3.3 BatchPublishEvents . . . 82

5.5.4 Publisher REST API . . . 83
5.5.4.1 Endpoints . . . 83
5.5.5 Event Consumer Service (Consumer API) . . . 85
5.5.5.1 Base URLs . . . 85
5.5.6 Consumer gRPC API . . . 85
5.5.6.1 Service Definition . . . 85
5.5.6.2 ConsumeNonDurableEvents . . . 86
5.5.6.3 ConsumeDurableEvents (Deprecated) . . . 88
5.5.6.4 ConsumeDurableEventsWithAcks . . . 89
5.5.6.5 CreateDurableQueue . . . 91
5.5.6.6 UpdateDurableQueue . . . 93
5.5.6.7 DeleteDurableQueue . . . 94
5.5.6.8 ManageDeadLetteredEvents . . . 94
5.5.7 Consumer REST API . . . 97
5.5.7.1 Endpoints . . . 97
5.5.8 Error Handling . . . 103
5.5.8.1 Common HTTP Status Codes . . . 103
5.5.8.2 Common gRPC Status Codes . . . 103
5.5.9 Protocol Buffer Files . . . 104
5.5.10 OpenAPI Specifications . . . 104
5.5.11 Rate Limits and Quotas . . . 104
5.5.12 Versioning . . . 104
5.5.13 Service Health . . . 104
5.5.14 Metrics and Monitoring . . . 105
5.5.15 Next Steps . . . 105

6 Quick Links **106**

7 Support and Contributing **107**

The Notifications Service is a distributed event messaging system that enables real-time event publishing and consumption across NVIDIA Omniverse services. It consists of two main components working together to provide a scalable, permission-aware event streaming infrastructure.

Chapter 1. What is the Notifications Service?

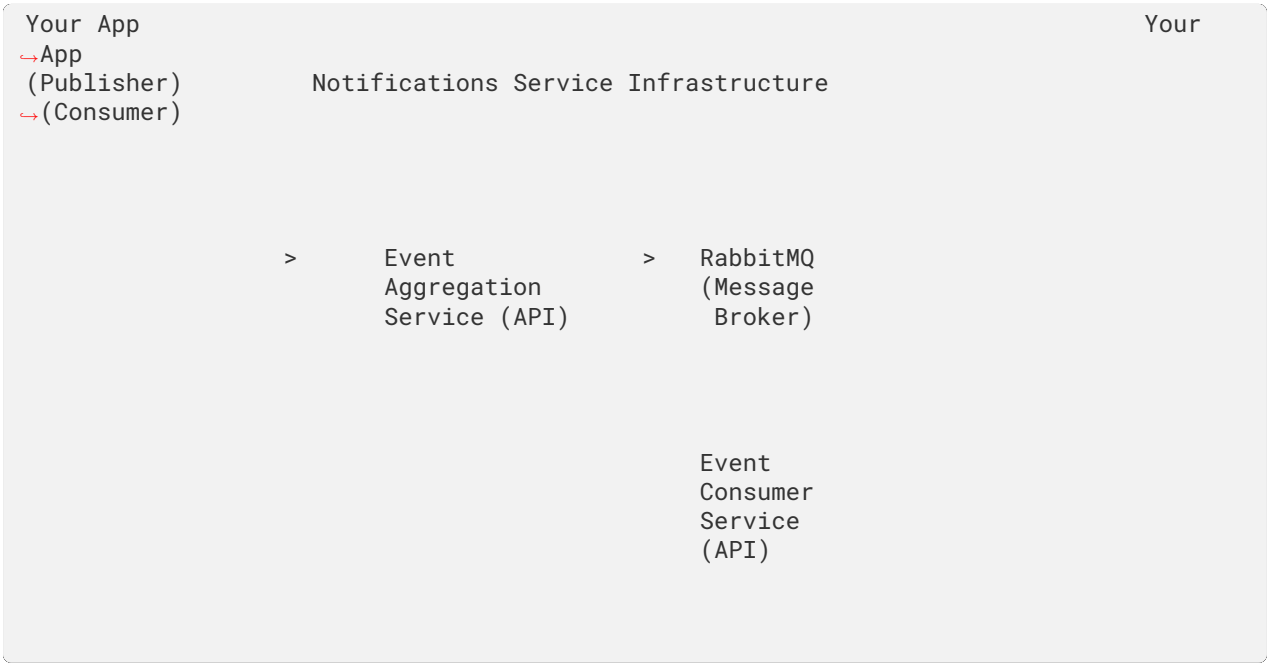
The Notifications Service provides a way for applications to:

- **Publish events** when significant actions occur (e.g., file created, user updated, workflow completed)
- **Consume events** in real-time to react to changes happening across the system
- **Filter events** to receive only the specific events your application cares about
- **Control access** through fine-grained permissions on both publishing and consuming

Chapter 2. Architecture Overview

The service consists of two independent services:

1. **Event Aggregation Service** (Publisher API) - Accepts events from publishers and routes them to RabbitMQ
2. **Event Consumer Service** (Consumer API) - Allows consumers to stream events from RabbitMQ with filtering and permissions



What this means:

- **Your App (Publisher)**: Any service you build that needs to publish events (e.g., a storage service publishing "file created" events)
- **Event Aggregation Service**: The Notifications Service API that receives your published events
- **RabbitMQ**: The message broker (managed by the Notifications Service)
- **Event Consumer Service**: The Notifications Service API that streams events to consumers
- **Your App (Consumer)**: Any service you build that needs to consume events (e.g., a thumbnail generator listening for file created events)

The **Notifications Service** = Event Aggregation Service + RabbitMQ + Event Consumer Service (the infrastructure in the box). You just interact with the two APIs.

Chapter 3. Key Concepts

3.1. Events

Events represent something that happened in your system. Each event has:

- **event_type**: A string identifying the type of event (e.g., “storage.file.created”)
- **message**: JSON payload containing event-specific data
- **occurred_at**: Timestamp when the event occurred
- **resource**: Optional path-like identifier for filtering (e.g., “/folder/subfolder/file.txt”)

3.2. Event Types and Message Contracts

Important: The Notifications Service is **event-type agnostic**. This means:

- The service does not know or validate the structure of your event types or message bodies
- Event types and message schemas are contracts between publishers and consumers
- Publishers and consumers must agree on:
  - What event type strings to use
  - What structure the message JSON should have
  - What the resource_id paths represent

For example, if you’re building a thumbnail service:

- Define your own event types like “thumbnail.created” or “thumbnail.updated”
- Document the message structure (e.g., `{"thumbnail_url": "...", "size": "..."}`)
- Ensure both publishers and consumers use the same schema

3.3. Durable vs Non-Durable Queues

The Consumer API supports two consumption modes:

Non-Durable Queues:

- Receive only events that occur while actively connected
- Lightweight, no setup required
- Best for real-time updates where missing messages is acceptable
- Automatically cleaned up when disconnected

Durable Queues:

- Persist events across disconnects, retaining them up to a configurable message TTL and a bounded maximum queue length
- Must be explicitly created before use
- Receive events from the time the queue was created, subject to those bounds
- Deliver at-least-once with bounded redelivery: a message that cannot be acknowledged within the redelivery limit — or that exceeds the message TTL or the maximum-length bound — is moved to a dead-letter queue rather than retained or redelivered indefinitely
- Best for critical event processing where events should not be silently missed within those bounds
- Require explicit cleanup when no longer needed, and are deleted automatically if left without a consumer for the deployment's idle window (24 hours by default)

Durability is therefore bounded, not unconditional. See *Bounded Redelivery and the Dead-Letter Queue* in the *Consuming Events* guide for the redelivery, TTL, and max-length behavior, *Durable Queue Lifetime* for the idle window and how a client recovers from it, and the same guide for guidance on when to use each queue type.

Dead-lettered messages are not a dead end: operators holding the appropriate permission can inspect them and replay, discard, or leave each one through a permission-gated, gRPC-only management RPC. See *Managing the Dead-Letter Queue (Operators)* in the *Consuming Events* guide.

3.4. Resource Filtering

Events can include a hierarchical `resource_id` (like a file path) that consumers can filter on:

- **Exact match**: Only events for a specific resource
- **Prefix match (lazy)**: Events for resources at a specific level
- **Prefix match (greedy)**: Events for resources and all sub-levels

Example:

```
Resource: "/projects/projectA/file.txt"

Filter "/projects/projectA/" (greedy)    Matches ✓
Filter "/projects/" (greedy)             Matches ✓
Filter "/projects/" (lazy)               No match   (file is nested too deep)
```

Chapter 4. Available APIs

Both services provide dual APIs:

- **gRPC API**: High-performance, strongly-typed, ideal for service-to-service communication
- **REST API**: HTTP/JSON-based, easier for testing and web clients, uses Server-Sent Events (SSE) for streaming

Chapter 5. Documentation Structure

5.1. Getting Started

This guide will help you quickly set up and start using the Notifications Service to publish and consume events.

5.1.1. Prerequisites

- Access to a deployed Notifications Service (Event Aggregation and Event Consumer services)
- Authentication credentials (JWT token or similar)
- For gRPC: Protocol Buffer compiler and gRPC libraries for your language
- For REST: HTTP client library (or just curl for testing)

5.1.2. Authentication

Both services require authentication. Include your bearer token in requests:

REST: Include the `Authorization` header:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

gRPC: Include the token in metadata:

```
# Python example
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]
```

5.1.3. Quick Example: Publish and Consume an Event

Let's walk through a complete example of publishing an event and consuming it.

5.1.3.1 Step 1: Publish an Event (REST)

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "event": {
      "event_type": "myapp.user.created",
      "message": {
        "user_id": "12345",
        "username": "john_doe",
        "email": "john@example.com"
      },
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/users/12345"
      }
    }
  }'
```

Response:

```
{
  "result": {
    "event": {
      "event_type": "myapp.user.created",
      "message": {
        "user_id": "12345",
        "username": "john_doe",
        "email": "john@example.com"
      },
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/users/12345"
      }
    },
    "success": true
  }
}
```

5.1.3.2 Step 2: Consume Events (REST with SSE)

Open a streaming connection to receive events:

```
curl -N https://your-consumer-service.example.com/api/v1beta/events/stream \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -G --data-urlencode 'filter_groups=[{
    "event_type": "myapp.user.created",
    "filters": []
  }]'
```

You'll receive Server-Sent Events as they occur:

```
event: event
data: {
  "event_type": "myapp.user.created",
  "principal_identity": "publisher@example.com",
  "occurred_at": "2024-10-16T14:30:00Z",
  "published_at": "2024-10-16T14:30:01Z",
  "message": {
    "user_id": "12345",
    "username": "john_doe",
    "email": "john@example.com"
  }
}
```

5.1.4. Quick Example: gRPC

5.1.4.1 Step 1: Generate Client Code

First, obtain the proto files from the released archive or protobuf registry.

Generate client code for your language:

```
# Python example
python -m grpc_tools.protoc \
  -I./protos \
  --python_out=. \
  --grpc_python_out=. \
  nvidia/omniverse/notifications/publisher/v1beta/event_publisher.proto \
  nvidia/omniverse/notifications/consumer/v1beta/event_consumer.proto
```

5.1.4.2 Step 2: Publish an Event (gRPC)

```
import grpc
from google.protobuf.timestamp_pb2 import Timestamp
from google.protobuf.struct_pb2 import Struct
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2_grpc

# Create channel with credentials
credentials = grpc.ssl_channel_credentials()
```

(continues on next page)

(continued from previous page)

```
channel = grpc.secure_channel(
    'your-aggregation-service.example.com:50051',
    credentials
)
stub = event_publisher_pb2_grpc.EventPublishingServiceStub(channel)

# Create event
message = Struct()
message.update({
    'user_id': '12345',
    'username': 'john_doe',
    'email': 'john@example.com'
})

timestamp = Timestamp()
timestamp.GetCurrentTime()

event = event_publisher_pb2.Event(
    event_type='myapp.user.created',
    message=message,
    occurred_at=timestamp,
    resource=event_publisher_pb2.EventResource(
        resource_id='/users/12345'
    )
)

# Publish event
request = event_publisher_pb2.PublishEventRequest(event=event)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]
response = stub.PublishEvent(request, metadata=metadata)

print(f"Published successfully: {response.result.success}")
```

5.1.4.3 Step 3: Consume Events (gRPC)

```
import grpc
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

# Create channel with credentials
credentials = grpc.ssl_channel_credentials()
channel = grpc.secure_channel(
    'your-consumer-service.example.com:50052',
    credentials
)
stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)

# Create filter groups
filter_group = event_consumer_pb2.FilterGroup(
    event_type='myapp.user.created'
)
# No filters added - receive all events of this type

# Create request stream
```

(continues on next page)

(continued from previous page)

```
def request_stream():
    request = event_consumer_pb2.ConsumeNonDurableEventsRequest()
    request.filter_groups.append(filter_group)
    yield request

# Consume events
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]
responses = stub.ConsumeNonDurableEvents(request_stream(), metadata=metadata)

for response in responses:
    # Process each batch of events
    for event in response.events:
        print(f"Received event: {event.event_type}")
        print(f"Message: {event.message}")
        print(f"Occurred at: {event.occurred_at}")
```

5.1.5. Common Patterns

5.1.5.1 1. Define Your Event Types

Before implementing, define your event type naming convention:

```
# Example event type definitions
USER_CREATED = "myapp.user.created"
USER_UPDATED = "myapp.user.updated"
USER_DELETED = "myapp.user.deleted"
FILE_UPLOADED = "myapp.file.uploaded"
WORKFLOW_COMPLETED = "myapp.workflow.completed"
```

5.1.5.2 2. Define Your Message Schemas

Document the structure of messages for each event type:

```
# Example: myapp.user.created message schema
{
    "user_id": str,      # Required: Unique user identifier
    "username": str,     # Required: User's username
    "email": str,        # Required: User's email
    "created_by": str,   # Optional: Who created this user
    "metadata": dict     # Optional: Additional metadata
}
```

5.1.5.3 3. Handle Errors

Both publishing and consuming can fail. Always handle errors appropriately:

```
# Publishing with error handling
try:
    response = stub.PublishEvent(request, metadata=metadata)
    if response.result.success:
        print("Event published successfully")
    else:
        print(f"Failed to publish: {response.result.failure_reason}")
except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.RESOURCE_EXHAUSTED:
        # Retry with exponential backoff
        print("Service temporarily unavailable, retrying...")
    elif e.code() == grpc.StatusCode.PERMISSION_DENIED:
        print("Permission denied - check your authorization")
    else:
        print(f"Unexpected error: {e.details()}")
```

5.1.6. Next Steps

Now that you've published and consumed your first events:

- Learn more about *Publishing Events* including batch publishing
- Explore *Consuming Events* including durable queues and filtering
- Understand *Permissions & Authorization* to secure your events
- Review the *API Reference* for complete API documentation

5.1.7. Troubleshooting

5.1.7.1 "Permission denied" errors

Make sure you have the appropriate permissions configured. See the *Permissions* guide.

5.1.7.2 "Service unavailable" errors

The service may be temporarily overloaded. Implement retry logic with exponential backoff.

5.1.7.3 Events not being received

- Check that your filter groups match the event types being published
- Verify your authentication token is valid
- Ensure the publisher is successfully publishing events

5.1.7.4 Connection drops

For non-durable queues, use the `reconnect_token` to reconnect to the same queue. That resumes the queue with the bindings it already holds, so a gRPC client that has changed filters mid-stream also re-sends the updates the server did not acknowledge — see *Updating Filters Dynamically*. For durable queues, reconnect with the same `queue_id`. If that reconnect returns `NOT_FOUND`, the queue was deleted after going unconsumed for the deployment's idle window — recreate it with the same `queue_id` and `filter_groups` and resume. See *Durable Queue Lifetime*.

5.2. Publishing Events

This guide covers how to publish events to the Notifications Service using both the REST API and gRPC API.

5.2.1. Overview

The Event Aggregation Service accepts events from publishers and routes them to consumers through RabbitMQ. Publishers can send individual events or batch multiple events for better performance.

5.2.2. Event Structure

Every event consists of the following fields:

Field	Type	Required	Description
`event_type`	string	Yes	A unique identifier for the type of event (e.g., "storage.file.created")
`message`	JSON object	Yes	The event payload - structure is defined by you
`occurred_at`	timestamp	Yes	When the event occurred (ISO 8601 format)
`resource`	object	No	Optional resource identifier for filtering

5.2.2.1 Event Type

The `event_type` is a string that identifies what kind of event this is. **You define your own event types** - the Notifications Service doesn't validate or enforce any particular naming scheme.

Recommended conventions:

- Use dot notation: `service.entity.action` (e.g., "storage.file.created")
- Be consistent across your organization
- Document your event types
- Use lowercase
- Be specific enough to allow filtering but not so specific that you have too many types

Examples:

```
storage.file.created
storage.file.deleted
storage.directory.created
project.workflow.completed
user.permission.updated
thumbnail.generated
```

5.2.2.2 Message

The `message` field contains the actual event payload as a JSON object. **The structure is entirely up to you** - define it based on what information consumers need.

Important: The Notifications Service does not validate or know anything about your message structure. It's a contract between your publishers and consumers.

Best practices:

- Document your message schema for each event type
- Include enough information for consumers to process the event
- Keep messages reasonably sized (avoid embedding large binary data)
- Use consistent field naming across event types
- Include identifiers that allow consumers to fetch more data if needed

Example message schemas:

```
// storage.file.created
{
  "file_name": "document.pdf",
  "file_size": 1024576,
  "mime_type": "application/pdf",
  "uploaded_by": "user@example.com",
  "checksum": "sha256:abc123..."
}

// project.workflow.completed
{
  "workflow_id": "wf-12345",
  "workflow_name": "Rendering Pipeline",
```

(continues on next page)

(continued from previous page)

```
  "status": "success",
  "duration_seconds": 3600,
  "output_location": "/renders/output.mp4"
}
```

5.2.2.3 Occurred At

An ISO 8601 timestamp indicating when the event occurred. Always use UTC.

Format: `YYYY-MM-DDTHH:MM:SSZ`

Example: `2024-10-16T14:30:00Z`

5.2.2.4 Resource (Optional)

The `resource` field contains a hierarchical identifier (like a file path) that consumers can use for filtering.

```
{
  "resource_id": "/folder/subfolder/file.txt"
}
```

Resource IDs should be:

- Path-like (slash-separated)
- Hierarchical (broader to more specific)
- Consistent across your event types

Examples:

```
/projects/projectA/file.txt
/users/12345
/workspaces/ws-001/render/output.mp4
```

If you omit the resource field, it's equivalent to including a resource that has `resource_id=""`.

5.2.3. Publishing a Single Event

5.2.3.1 REST API

Endpoint: `POST /api/v1beta/events`

Example:

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "event": {
      "event_type": "storage.file.created",
      "message": {
```

(continues on next page)

(continued from previous page)

```
        "file_name": "document.pdf",
        "file_size": 1024576,
        "mime_type": "application/pdf",
        "uploaded_by": "user@example.com"
      },
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/uploads/documents/document.pdf"
      }
    }
  }'
```

Response (Success):

```
{
  "result": {
    "event": {
      "event_type": "storage.file.created",
      "message": {
        "file_name": "document.pdf",
        "file_size": 1024576,
        "mime_type": "application/pdf",
        "uploaded_by": "user@example.com"
      },
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/uploads/documents/document.pdf"
      }
    },
    "success": true
  }
}
```

Response (Failure):

```
{
  "result": {
    "event": { /* event details */ },
    "success": false,
    "failure_reason": "Unable to connect to message broker"
  }
}
```

5.2.3.2 gRPC API

RPC: `PublishEvent`

Python Example:

```
import grpc
from google.protobuf.timestamp_pb2 import Timestamp
from google.protobuf.struct_pb2 import Struct
from datetime import datetime
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2_grpc
```

(continues on next page)

(continued from previous page)

```
# Setup channel and stub
channel = grpc.secure_channel(
    'your-aggregation-service.example.com:50051',
    grpc.ssl_channel_credentials()
)
stub = event_publisher_pb2_grpc.EventPublishingServiceStub(channel)

# Create message
message = Struct()
message.update({
    'file_name': 'document.pdf',
    'file_size': 1024576,
    'mime_type': 'application/pdf',
    'uploaded_by': 'user@example.com'
})

# Create timestamp
timestamp = Timestamp()
timestamp.FromDatetime(datetime(2024, 10, 16, 14, 30, 0))

# Create event
event = event_publisher_pb2.Event(
    event_type='storage.file.created',
    message=message,
    occurred_at=timestamp,
    resource=event_publisher_pb2.EventResource(
        resource_id='/uploads/documents/document.pdf'
    )
)

# Publish
request = event_publisher_pb2.PublishEventRequest(event=event)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

try:
    response = stub.PublishEvent(request, metadata=metadata)
    if response.result.success:
        print("Event published successfully")
    else:
        print(f"Failed: {response.result.failure_reason}")
except grpc.RpcError as e:
    print(f"RPC failed: {e.code()}: {e.details()}")
```

Go Example:

```
package main

import (
    "context"
    "time"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials"
    "google.golang.org/protobuf/types/known/structpb"
    "google.golang.org/protobuf/types/known/timestamppb"
```

(continues on next page)

(continued from previous page)

```
    pb "nvidia/omniverse/notifications/publisher/v1beta"
)

func main() {
    // Setup connection
    creds := credentials.NewTLS(&tls.Config{})
    conn, err := grpc.Dial(
        "your-aggregation-service.example.com:50051",
        grpc.WithTransportCredentials(creds),
    )
    if err != nil {
        panic(err)
    }
    defer conn.Close()

    client := pb.NewEventPublishingServiceClient(conn)

    // Create message
    message, _ := structpb.NewStruct(map[string]interface{}{
        "file_name":   "document.pdf",
        "file_size":   1024576,
        "mime_type":   "application/pdf",
        "uploaded_by": "user@example.com",
    })

    // Create event
    event := &pb.Event{
        EventType:  "storage.file.created",
        Message:    message,
        OccurredAt: timestamppb.New(time.Now()),
        Resource: &pb.EventResource{
            ResourceId: "/uploads/documents/document.pdf",
        },
    }

    // Publish
    ctx := context.Background()
    // Add auth metadata
    ctx = metadata.AppendToOutgoingContext(ctx, "authorization", "Bearer "+YOUR_TOKEN)

    resp, err := client.PublishEvent(ctx, &pb.PublishEventRequest{
        Event: event,
    })

    if err != nil {
        panic(err)
    }

    if resp.Result.Success {
        println("Event published successfully")
    } else {
        println("Failed:", resp.Result.FailureReason)
    }
}
```

5.2.4. Batch Publishing Multiple Events

When you need to publish multiple events, use batch publishing for better performance. Events in a batch are published in parallel.

5.2.4.1 REST API

Endpoint: `POST /api/v1beta/events/batch`

Example:

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events/batch \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "events": [
      {
        "event_type": "storage.file.created",
        "message": {
          "file_name": "document1.pdf"
        },
        "occurred_at": "2024-10-16T14:30:00Z",
        "resource": {
          "resource_id": "/uploads/document1.pdf"
        }
      },
      {
        "event_type": "storage.file.created",
        "message": {
          "file_name": "document2.pdf"
        },
        "occurred_at": "2024-10-16T14:30:01Z",
        "resource": {
          "resource_id": "/uploads/document2.pdf"
        }
      },
      {
        "event_type": "storage.file.created",
        "message": {
          "file_name": "document3.pdf"
        },
        "occurred_at": "2024-10-16T14:30:02Z",
        "resource": {
          "resource_id": "/uploads/document3.pdf"
        }
      }
    ]
  }'
```

Response:

```
{
  "results": [
    {
      "event": { /* event 1 */ },
      "success": true
```

(continues on next page)

(continued from previous page)

```
    },
    {
      "event": { /* event 2 */ },
      "success": true
    },
    {
      "event": { /* event 3 */ },
      "success": false,
      "failure_reason": "Failed to publish to exchange"
    }
  ]
}
```

Important: Always check the `results` array - some events may succeed while others fail.

5.2.4.2 gRPC API

RPC: `BatchPublishEvents`

Python Example:

```python
import grpc
from google.protobuf.timestamp_pb2 import Timestamp
from google.protobuf.struct_pb2 import Struct
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2_grpc

# Setup channel and stub
channel = grpc.secure_channel(
    'your-aggregation-service.example.com:50051',
    grpc.ssl_channel_credentials()
)
stub = event_publisher_pb2_grpc.EventPublishingServiceStub(channel)

# Create multiple events
events = []
for i in range(3):
    message = Struct()
    message.update({
        'file_name': f'document{i+1}.pdf'
    })

    timestamp = Timestamp()
    timestamp.GetCurrentTime()

    event = event_publisher_pb2.Event(
        event_type='storage.file.created',
        message=message,
        occurred_at=timestamp,
        resource=event_publisher_pb2.EventResource(
            resource_id=f'/uploads/document{i+1}.pdf'
        )
    )
    events.append(event)
```

(continues on next page)

(continued from previous page)

```
# Batch publish
request = event_publisher_pb2.BatchPublishEventsRequest()
for event in events:
    request.events.append(event)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

try:
    response = stub.BatchPublishEvents(request, metadata=metadata)

    # Check each result
    for idx, result in enumerate(response.results):
        if result.success:
            print(f"Event {idx+1} published successfully")
        else:
            print(f"Event {idx+1} failed: {result.failure_reason}")

except grpc.RpcError as e:
    print(f"RPC failed: {e.code()}: {e.details()}")
```

5.2.5. Error Handling

5.2.5.1 Common Error Codes

HTTP Status / gRPC Code	Meaning	Action
401 / UNAUTHENTICATED	Authentication failed	Check your token is valid and not expired
403 / PERMISSION_DENIED	Not authorized to publish this event type	Check permissions configuration
400 / INVALID_ARGUMENT	Invalid request format	Check your event structure matches the schema
503 / RESOURCE_EXHAUSTED	Service temporarily overloaded	Retry with exponential backoff
500 / INTERNAL	Server error	Check service health, contact support

5.2.5.2 Retry Logic

Implement retry logic with exponential backoff for transient failures:

```
import time
from typing import Callable

def publish_with_retry(
    publish_func: Callable,
    max_retries: int = 3,
```

(continues on next page)

(continued from previous page)

```
    initial_delay: float = 1.0
) -> bool:
    """
    Retry publishing with exponential backoff.
    """
    delay = initial_delay

    for attempt in range(max_retries):
        try:
            response = publish_func()

            if response.result.success:
                return True
            else:
                print(f"Publish failed: {response.result.failure_reason}")
                return False

        except grpc.RpcError as e:
            if e.code() == grpc.StatusCode.RESOURCE_EXHAUSTED:
                if attempt < max_retries - 1:
                    print(f"Service overloaded, retrying in {delay}s...")
                    time.sleep(delay)
                    delay *= 2  # Exponential backoff
                else:
                    print("Max retries exceeded")
                    return False
            else:
                # Don't retry for other errors
                print(f"Non-retriable error: {e.code()}")
                return False

    return False

# Usage
def publish():
    return stub.PublishEvent(request, metadata=metadata)

success = publish_with_retry(publish)
```

5.2.6. Best Practices

5.2.6.1 1. Choose Appropriate Event Granularity

Too granular:

```
# Publishing separate events for every field change
publish_event("user.email.changed", {...})
publish_event("user.name.changed", {...})
publish_event("user.phone.changed", {...})
```

Better:

```
# Single event for the update with changed fields in message
publish_event("user.updated", {
    "user_id": "123",
    "changed_fields": ["email", "name", "phone"],
    "new_values": {...}
})
```

5.2.6.2 2. Include Timestamps Accurately

Use the actual time the event occurred, not when you're publishing it:

```
# Good
occurred_at = user_created_timestamp

# Bad
occurred_at = datetime.utcnow()  # Time of publishing, not event occurrence
```

5.2.6.3 3. Use Resource IDs Consistently

Establish conventions for resource IDs across your organization:

```
# Good - consistent path structure
"/users/{user_id}"
"/projects/{project_id}/files/{file_id}"
"/workspaces/{workspace_id}"

# Bad - inconsistent
"user-123"
"project_456/file789"
"/workspace:ws-001"
```

5.2.6.4 4. Batch When Possible

If publishing multiple events at once, use batch publishing:

```
# Good - batch publish
events = [create_event(file) for file in uploaded_files]
batch_publish(events)

# Bad - individual publishes in a loop
for file in uploaded_files:
    publish_event(create_event(file))  # Slower
```

5.2.6.5 5. Handle Partial Batch Failures

When batch publishing, some events may succeed while others fail:

```
response = stub.BatchPublishEvents(request, metadata=metadata)

failed_events = [
    result.event for result in response.results
    if not result.success
]

if failed_events:
    # Log failures, retry, or handle them appropriately
    log_failed_events(failed_events)
    # Maybe retry just the failed events
    retry_publish(failed_events)
```

5.2.6.6 6. Keep Messages Reasonably Sized

Don't embed large payloads in events:

```
# Bad - embedding large binary data
publish_event("image.processed", {
    "image_data": base64_encode(large_image)  # Don't do this!
})

# Good - reference the data
publish_event("image.processed", {
    "image_url": "https://storage.example.com/images/123.jpg",
    "thumbnail_url": "https://storage.example.com/thumbs/123.jpg",
    "size_bytes": 1024576
})
```

5.2.6.7 7. Document Your Events

Maintain documentation of your event types and message schemas:

```
"""
Event Type: storage.file.created

Description: Published when a new file is uploaded to storage

Message Schema:
{
    "file_name": str,           # Name of the file
    "file_size": int,           # Size in bytes
    "mime_type": str,           # MIME type of the file
    "uploaded_by": str,         # Email of uploader
    "checksum": str,            # SHA-256 checksum (optional)
    "metadata": dict            # Additional metadata (optional)
}

Resource ID Format: /path/to/file
Example: /uploads/2024/10/document.pdf
"""
```

5.2.7. Performance Considerations

- **Batch publishing**: Use for multiple events (faster than individual publishes)
- **Connection pooling**: Reuse gRPC channels/connections instead of creating new ones for each publish
- **Async publishing**: If your language/framework supports it, publish asynchronously
- **Message size**: Keep messages under 100KB for optimal performance

5.2.8. Next Steps

- Learn how to *Consume Events*
- Understand *Permissions & Authorization* for publishing
- Review the *API Reference* for complete API details

5.3. Consuming Events

This guide covers how to consume events from the Notifications Service using both the REST API and gRPC API, including detailed information about durable and non-durable queues.

5.3.1. Overview

The Event Consumer Service allows clients to stream events in real-time. You can filter events by event type and resource ID, and choose between durable and non-durable consumption modes.

5.3.2. Durable vs Non-Durable Queues

One of the most important decisions when consuming events is whether to use a **durable queue** or a **non-durable queue**. Understanding the difference is critical for building reliable applications.

5.3.2.1 Non-Durable Queues

What they are:

- Temporary queues created automatically when you start consuming
- Only receive events that occur **while you are connected**
- Automatically deleted after disconnection (after a brief TTL)
- No setup required - just start streaming

When to use:

- Real-time notifications in UI applications
- Live dashboards and monitoring displays
- Event logging where missing some events is acceptable
- Prototyping and development
- Scenarios where events are only relevant “right now”

Advantages:

- Simple - no queue management needed
- Lightweight - no persistent storage
- Automatic cleanup
- Good for applications with many concurrent users (each gets their own temporary queue)

Disadvantages:

- **Events occurring while disconnected are lost**
- Not suitable for critical event processing

Example use cases:

```
✓ Showing "Someone just uploaded a file" notifications in a web UI
✓ Live activity feed in a dashboard
✓ Real-time log streaming during development
  Processing every file upload for thumbnail generation (use durable)
  Triggering critical workflows (use durable)
  Syncing data between services (use durable)
```

5.3.2.2 Durable Queues

What they are:

- Persistent queues that must be explicitly created before use
- Store events even when you’re disconnected
- Receive **all events from the time the queue was created**
- Should be explicitly deleted when no longer needed
- Deleted automatically by the service if left without a consumer for the deployment’s idle window (24 hours by default) — see *Durable Queue Lifetime*

When to use:

- Critical event processing where no events can be missed
- Microservices that need to process every event
- Batch processing workflows
- Systems that may have downtime for deployments
- Event-driven architectures where events trigger actions

Advantages:

- **No events are lost** - all events since queue creation are preserved
- Can disconnect and reconnect without losing messages

- Events are queued up during downtime
- Reliable for building event-driven systems

Disadvantages:

- Requires explicit queue lifecycle management (create/delete)
- Consumes storage resources
- Need to persist **both the queue_id and the filter_groups** in your application configuration
- Must remember to delete queues when done
- Not consuming for longer than the idle window deletes the queue, and your client must recreate it
- A client-chosen `queue_id` must be unique across the whole deployment, must match `^[A-Za-z0-9._-]{1,200}$`, and any principal holding a durable-queue permission can reach a queue whose id it knows

Example use cases:

```
✓ Thumbnail generation service processing every file upload
✓ Notification delivery service sending emails for events
✓ Data synchronization between services
✓ Audit logging where every event must be recorded
  UI notifications (non-durable is simpler)
  Temporary development testing (non-durable is easier)
```

5.3.2.3 Decision Matrix

Scenario	Recommended Choice	Reason
UI real-time notifications	**Non-durable**	Users only care about what's happening now
File processing pipeline	**Durable**	Must process every file
Live dashboard	**Non-durable**	Current state is what matters
Email notification service	**Durable**	Must send every notification
Development/testing	**Non-durable**	Easier, no cleanup needed
Production microservice	**Durable**	Can't miss events during restarts
Webhook relay	**Durable**	Must deliver all webhooks
Activity feed	**Non-durable**	Only recent activity matters

5.3.2.4 Key Principle

If missing an event would break your application or cause data inconsistency, use a durable queue. If events are only relevant "in the moment," use a non-durable queue.

5.3.3. Consuming with Non-Durable Queues

5.3.3.1 REST API

Endpoint: `GET /api/v1beta/events/stream`

The REST API uses Server-Sent Events (SSE) for streaming.

Basic Example (No Filtering):

```
# Consume all events of all types
curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

With Event Type Filtering:

```
# Create filter groups as JSON
FILTERS='[
  {
    "event_type": "storage.file.created",
    "filters": []
  }
]'

# URL encode and pass as query parameter
curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -G --data-urlencode "filter_groups=$FILTERS"
```

With Resource Filtering:

```
# Only receive events for files in /uploads/ directory and subdirectories
FILTERS='[
  {
    "event_type": "storage.file.created",
    "filters": [
      {
        "filter_type": "starts_with_greedy",
        "resource_id": "/uploads/"
      }
    ]
  }
]'

curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -G --data-urlencode "filter_groups=$FILTERS"
```

Response Format:

```
event: connected
id: my-queue-id
data: {"message":"Connection established","reconnect_token":"my-queue-id"}

event: event
data: {"event_type":"storage.file.created","principal_identity":"user@example.com",
↪"occurred_at":"2024-10-16T14:30:00Z","published_at":"2024-10-16T14:30:01Z","message
↪":{"file_name":"doc.pdf"}}

event: event
data: {"event_type":"storage.file.created","principal_identity":"user2@example.com",
↪"occurred_at":"2024-10-16T14:31:00Z","published_at":"2024-10-16T14:31:01Z","message
↪":{"file_name":"image.png"}}
```

The initial `connected` event includes the `reconnect_token` (which is the queue ID). Save this token for reconnection.

Presenting the token reattaches to the queue it names, which is what lets you reconnect after a drop and share one queue across parallel clients; consumers on the same queue split its events between them. When several clients share a queue this way, at most one of them may change its filters, and that client alone can follow the acknowledgement-driven replay recipe described under **Updating Filters Dynamically**.

The token identifies a queue. It is not a secret and it grants no access on its own: presenting it requires a valid bearer token, and every event delivered to you is authorized against your own permissions, so you receive only what you are entitled to receive whichever queue you attach to. What the token does is name the queue you join — and because consumers on one queue split its events, keeping the token inside the application that received it is what ensures only your own authorized consumers share, and split, that queue's events.

Send the token in the `Last-Event-ID` request header; the `reconnect_token` query parameter is deprecated in favour of it but is still accepted and carries the token the same way. The header carries every operation the query parameter does, including a filter update, so there is nothing you need the query parameter for. Prefer the header: a query parameter travels in the URL, and URLs are recorded by proxies, gateways, and browser history along the way, while a header is not.

A token you send back must be 1 to 200 printable ASCII characters — no spaces, no control characters, nothing non-ASCII — in the header and in the query parameter alike; anything else is refused with 422 `Unprocessable Entity` (`INVALID_ARGUMENT` on gRPC), naming the field and the rule and quoting none of the token. The tokens this service issues are random identifiers well inside that rule, so echoing back what you were given always passes it.

Python Example with SSE Client:

```python
import sseclient
import requests
import json

url = "https://your-consumer-service.example.com/api/v1beta/events/stream"
headers = {
    "Authorization": f"Bearer {YOUR_TOKEN}",
    "Accept": "text/event-stream"
}

# Define filters
filters = [
    {
        "event_type": "storage.file.created",
```

(continues on next page)

(continued from previous page)

```
        "filters": [
            {
                "filter_type": "starts_with_greedy",
                "resource_id": "/uploads/"
            }
        ]
    }
]

params = {"filter_groups": json.dumps(filters)}

# Stream events
response = requests.get(url, headers=headers, params=params, stream=True)
client = sseclient.SSEClient(response)

reconnect_token = None

for event in client.events():
    if event.event == "connected":
        connected_data = json.loads(event.data)
        reconnect_token = connected_data['reconnect_token']
        print(f"Connected, reconnect token: {reconnect_token}")
    elif event.event == "event":
        data = json.loads(event.data)
        print(f"Received: {data['event_type']}")
        print(f"Message: {data['message']}")
```

Reconnecting After Disconnection:

If your connection drops, use the reconnect token to reconnect to the same queue and avoid missing events. Send back the token the server issued, verbatim, in the `Last-Event-ID` header:

```
curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Last-Event-ID: abc123xyz"
```

A browser's built-in SSE reconnect sends this header for you, using the `id:` field of the last event it received.

Updating Filters Dynamically (REST):

You can update filters on a non-durable queue while it continues to exist. This is useful when you need to change what events you're listening for without losing the queue. To update filters, make a new request carrying the token, the new `filter_groups`, and the `previous_filter_groups`.

Sending `previous_filter_groups` is what makes a request a filter update, whichever carrier the token arrives in — the `Last-Event-ID` header and the deprecated `reconnect_token` query parameter both work. A request that carries the token alone is a plain reconnect and resumes the stream on the filters it already had.

A browser's built-in SSE reconnect arrives at the same result by the header path. It replays the URL your stream was opened on and sends the token in the `Last-Event-ID` header; for a stream opened as a fresh connect that URL carries the original `filter_groups` and no `previous_filter_groups` — so the automatic reconnect resumes the stream as it was rather than being read as an incomplete filter update. If instead you opened the `EventSource` on a filter-update URL, the browser replays `previous_filter_groups` too and each automatic reconnect re-runs that same update, which is harmless: applying the same before-and-after pair to a queue already in that state changes nothing.

```
# Original filters we were using
PREV_FILTERS='[
  {
    "event_type": "storage.file.created",
    "filters": [
      {
        "filter_type": "starts_with_greedy",
        "resource_id": "/uploads/"
      }
    ]
  }
]'

# New filters we want to use
NEW_FILTERS='[
  {
    "event_type": "storage.file.created",
    "filters": [
      {
        "filter_type": "starts_with_greedy",
        "resource_id": "/uploads/"
      }
    ]
  },
  {
    "event_type": "storage.file.deleted",
    "filters": [
      {
        "filter_type": "starts_with_greedy",
        "resource_id": "/uploads/"
      }
    ]
  }
]'

# Reconnect with updated filters
curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Last-Event-ID: abc123xyz" \
  -G --data-urlencode "filter_groups=$NEW_FILTERS" \
  --data-urlencode "previous_filter_groups=$PREV_FILTERS"
```

Python Example with Filter Updates:

```
import sseclient
import requests
import json

def consume_with_filter_updates():
    url = "https://your-consumer-service.example.com/api/v1beta/events/stream"
    headers = {
        "Authorization": f"Bearer {YOUR_TOKEN}",
        "Accept": "text/event-stream"
    }

    # Initial filters
    current_filters = [
```

(continues on next page)

(continued from previous page)

```
        {
            "event_type": "storage.file.created",
            "filters": [
                {"filter_type": "starts_with_greedy", "resource_id": "/uploads/"}
            ]
        }
    ]

    params = {"filter_groups": json.dumps(current_filters)}
    reconnect_token = None

    # Start consuming
    response = requests.get(url, headers=headers, params=params, stream=True)
    client = sseclient.SSEClient(response)

    for event in client.events():
        if event.event == "connected":
            connected_data = json.loads(event.data)
            reconnect_token = connected_data['reconnect_token']
        elif event.event == "event":
            data = json.loads(event.data)
            process_event(data)

            # Example: Update filters after some condition
            if should_update_filters():
                # Close current connection
                response.close()

                # Define new filters
                new_filters = [
                    {
                        "event_type": "storage.file.created",
                        "filters": [
                            {"filter_type": "starts_with_greedy",
                             "resource_id": "/uploads/"}
                        ]
                    },
                    {
                        "event_type": "storage.file.deleted",
                        "filters": [
                            {"filter_type": "starts_with_greedy",
                             "resource_id": "/uploads/"}
                        ]
                    }
                ]

                # Reconnect with updated filters. The token goes in the
                # Last-Event-ID header; sending previous_filter_groups is
                # what makes this an update rather than a plain reconnect.
                params = {
                    "filter_groups": json.dumps(new_filters),
                    "previous_filter_groups": json.dumps(current_filters)
                }
                reconnect_headers = {**headers,
                                     "Last-Event-ID": reconnect_token}
```

(continues on next page)

(continued from previous page)

```
                response = requests.get(url, headers=reconnect_headers,
                                        params=params, stream=True)
                client = sseclient.SSEClient(response)

                # Update our tracking of current filters
                current_filters = new_filters
```

Important Notes on Updating Filters (REST):

- When updating filters, you **must** provide both `filter_groups` (new filters) and `previous_filter_groups` (old filters). Providing `previous_filter_groups` is what marks the request as an update
- The reconnect token identifies which queue to update. Send it in the `Last-Event-ID` header (or in the deprecated `reconnect_token` query parameter); either carrier performs the update
- When the reconnect token is sent in the `Last-Event-ID` header without `previous_filter_groups`, any `filter_groups` on the request are ignored and the stream resumes on the filters it already had. Include `previous_filter_groups` to update the filters. That tolerance is specific to the header, which a browser's automatic reconnect replays: a token sent in the deprecated `reconnect_token` query parameter alongside `filter_groups` but without `previous_filter_groups` is an incomplete filter update and is rejected with `400`, naming the missing field
- The queue must already exist (you cannot update filters before making an initial consume request)
- If `filter_groups` equals `previous_filter_groups`, no update is performed
- There is no separate acknowledgement on this surface, and none is needed: a REST filter update is applied before the stream's `connected` event is sent, so the arrival of that event confirms the update. The gRPC surface, where updates are sent mid-stream, acknowledges each one it processes

5.3.3.2 gRPC API

RPC: `ConsumeNonDurableEvents` (bidirectional streaming)

Python Example:

```
import grpc
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

# Setup channel and stub
channel = grpc.secure_channel(
    'your-consumer-service.example.com:50052',
    grpc.ssl_channel_credentials()
)
stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)

# Define filters
filter_group = event_consumer_pb2.FilterGroup(
    event_type='storage.file.created'
)
resource_filter = event_consumer_pb2.ResourceFilter(
```

(continues on next page)

(continued from previous page)

```
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
)
filter_group.filters.append(resource_filter)

# Create request stream
def request_stream():
    # Send initial request with filters
    request = event_consumer_pb2.ConsumeNonDurableEventsRequest()
    request.filter_groups.append(filter_group)
    yield request
    # Keep stream open (or send updated filters later)

# Start consuming
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]
responses = stub.ConsumeNonDurableEvents(request_stream(), metadata=metadata)

reconnect_token = None
server_acks_filter_updates = None  # Unknown until the first response arrives
confirmed_filters = None  # The set the server last acknowledged

try:
    for response in responses:
        # Save reconnect token for potential reconnection
        if response.reconnect_token:
            reconnect_token = response.reconnect_token

        if server_acks_filter_updates is None:
            # True on every response of a stream served by a server that
            # acknowledges the filter sets its requests name. It describes
            # the server, not this response, so reading it off the first
            # response is enough -- and it is what picks your recovery path.
            server_acks_filter_updates = response.supports_filter_update_acks
            if server_acks_filter_updates:
                print("Filter updates are acknowledged: after a drop, "
                      "reconnect with the token and replay every update "
                      "that was not acknowledged")
            else:
                print("Filter updates are not acknowledged: after a drop, "
                      "open a fresh stream with filter_groups and no "
                      "reconnect_token")

        # A response carrying filter_update_ack reports that the server acted
        # on the filter set named by one of your requests; it carries no
        # events. The first response carries one when the opening request
        # named a set, as this fresh connect does. applied_filter_groups is a
        # verbatim copy of that request's filter_groups, and unchanged is
        # True when its filter_groups equalled its previous_filter_groups as
        # you sent them, so the server issued no bind or unbind call.
        if response.HasField('filter_update_ack'):
            # Name this set in previous_filter_groups on your next update,
            # and treat every update you sent after it as outstanding until
            # its own acknowledgement arrives.
            confirmed_filters = list(
                response.filter_update_ack.applied_filter_groups
            )
```

(continues on next page)

(continued from previous page)

```
            print(f"Filter update acknowledged: {confirmed_filters} "
                  f"(unchanged={response.filter_update_ack.unchanged})")
            continue

        # Process events
        for event in response.events:
            print(f"Event type: {event.event_type}")
            print(f"Occurred at: {event.occurred_at}")
            print(f"Message: {event.message}")
            print(f"Published by: {event.principal_identity}")
            print("---")

except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.UNAUTHENTICATED:
        print("Token expired - reconnect with new token")
        # Reconnect with new token and reconnect_token
```

Reconnecting with Token:

A reconnect that carries only the token resumes the queue with the bindings it already holds, and the first response carries no `filter_update_ack` — the server keeps no filter state and cannot report a set it was not told. A reconnect may instead carry the token together with both `filter_groups` and `previous_filter_groups`, which is a filter update applied before the stream yields anything and acknowledged on that first response. A reconnect that names `filter_groups` without `previous_filter_groups` is neither: it is an incomplete filter update and is rejected with `INVALID_ARGUMENT`. A client that has changed filters since the set the server last acknowledged replays the outstanding updates after reconnecting; see **Updating Filters Dynamically** below.

```
def request_stream_reconnect(reconnect_token):
    request = event_consumer_pb2.ConsumeNonDurableEventsRequest(
        reconnect_token=reconnect_token
    )
    yield request

# Reconnect to same queue
responses = stub.ConsumeNonDurableEvents(
    request_stream_reconnect(reconnect_token),
    metadata=new_metadata
)
```

Updating Filters Dynamically:

The bidirectional stream allows you to update filters while consuming. When sending an update request, you must include both the new filters in `filter_groups` and the previous filters in `previous_filter_groups`.

A server that sets `supports_filter_update_acks` sends one response carrying a `filter_update_ack` for each update it processes. Receiving it means the server is done with that request: any bind and unbind calls the request led to have returned — it does not report how many calls that was, and the answer may be none. Track the updates you send and mark each one confirmed when its acknowledgement arrives: that is what lets you resume this stream with the `reconnect_token` after a drop, by re-sending in order each update you sent but did not see acknowledged, each naming in `previous_filter_groups` the set it replaced.

```
import threading
import queue
```

(continues on next page)

(continued from previous page)

```
# Track current filters so we can send them as previous_filter_groups
current_filters = []

def request_stream_dynamic(filter_queue, initial_filters):
    global current_filters

    # Send initial filters
    initial_request = event_consumer_pb2.ConsumeNonDurableEventsRequest()
    for fg in initial_filters:
        initial_request.filter_groups.append(fg)
    current_filters = initial_filters
    yield initial_request

    # Wait for filter updates
    while True:
        new_filters = filter_queue.get()  # Blocks until new filters available
        if new_filters is None:  # Sentinel to stop
            break

        # Create update request with both new and previous filters
        update_request = event_consumer_pb2.ConsumeNonDurableEventsRequest()
        for fg in new_filters:
            update_request.filter_groups.append(fg)
        for fg in current_filters:
            update_request.previous_filter_groups.append(fg)

        # Update our tracking of current filters
        current_filters = new_filters
        yield update_request

# Initial filters
initial_filter_group = event_consumer_pb2.FilterGroup(
    event_type='storage.file.created'
)
resource_filter = event_consumer_pb2.ResourceFilter(
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
)
initial_filter_group.filters.append(resource_filter)
initial_filters = [initial_filter_group]

filter_queue = queue.Queue()
responses = stub.ConsumeNonDurableEvents(
    request_stream_dynamic(filter_queue, initial_filters),
    metadata=metadata
)

# Later, update filters to add a new event type
new_filter_group_1 = event_consumer_pb2.FilterGroup(
    event_type='storage.file.created'
)
new_filter_group_1.filters.append(event_consumer_pb2.ResourceFilter(
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
))
```

(continues on next page)

(continued from previous page)

```
new_filter_group_2 = event_consumer_pb2.FilterGroup(
    event_type='storage.file.deleted'
)
new_filter_group_2.filters.append(event_consumer_pb2.ResourceFilter(
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
))

# Send updated filters - the request_stream_dynamic function will
# automatically include the previous filters
filter_queue.put([new_filter_group_1, new_filter_group_2])

# Reading the acknowledgements on the response side. Each acknowledgement
# names the update it confirms by echoing that request's filter_groups, so a
# client knows which of the updates it sent the server has processed.
for response in responses:
    if response.HasField('filter_update_ack'):
        ack = response.filter_update_ack
        # unchanged is True when the acknowledged request's filter_groups
        # equalled its previous_filter_groups as you sent them, so the server
        # issued no bind or unbind call.
        print(f"Applied: {list(ack.applied_filter_groups)} "
              f"unchanged={ack.unchanged}")
        continue

    for event in response.events:
        print(f"Event type: {event.event_type}")
```

Important Notes on Updating Filters (gRPC):

- When updating filters, you **must** provide both `filter_groups` (new filters) and `previous_filter_groups` (old filters)
- If `filter_groups` equals `previous_filter_groups` as you sent them, the server issues no bind or unbind call, and the request is acknowledged with `unchanged` set to `true`
- The server uses `previous_filter_groups` to determine which bindings to remove when updating the queue
- `applied_filter_groups` is a verbatim copy of the acknowledged request's `filter_groups`: the server holds no filter state and cannot read the queue's bindings back from the broker, so it echoes what you sent rather than reporting what the broker holds
- `unchanged` describes the request the server received rather than the queue's state. It does not confirm that the queue's bindings match `applied_filter_groups`, and a `false` value does not promise that any bind or unbind call was issued — a request that re-sends the same groups reordered, or with a group repeated, differs as sent and still changes no bindings
- The first response on the stream carries an acknowledgement when the opening request named a filter set the server acted on: a fresh connect, acknowledged with `unchanged` `false`, and a reconnect naming `filter_groups` and `previous_filter_groups` together, acknowledged with `unchanged` set according to whether the two were equal as sent. A reconnect carrying only the token names no set and is not acknowledged, and a reconnect naming `filter_groups` without `previous_filter_groups` is rejected with `INVALID_ARGUMENT`
- A mid-stream request that names neither `filter_groups` nor `previous_filter_groups` is ignored and never acknowledged; a rejected update ends the stream with `INVALID_ARGUMENT`

and is not acknowledged either. Nor is an update still in flight when the stream ends with a terminal error, as it does when the broker cancels the consumer (see *Handle Disconnections Gracefully*): the stream ends and the update is never acknowledged

- Acknowledgements are ordered with respect to one another, in the order the updates were sent, but an acknowledgement and the event responses around it are interleaved in no guaranteed order
- Replaying more than one unacknowledged update after a reconnect walks the queue back through each intermediate filter set in turn, and applying a set unbinds every resource that set drops, so an event published for a resource that an earlier replayed update unbinds is lost until a later replayed update binds it again
- Exactly one client per queue may change filters. Other clients sharing the `reconnect_token` may consume from the queue, but must not send filter updates: each client computes `previous_filter_groups` from acknowledgements that describe only its own updates, so two clients updating one queue walk its bindings to a set neither asked for
- Against a server that does not set `supports_filter_update_acks` there is no safe way to resume a changed filter set by token; open a fresh stream with `filter_groups` and no `reconnect_token` instead

5.3.4. Consuming with Durable Queues

Durable queues require a three-step process: create, consume, delete.

5.3.4.1 Step 1: Create a Durable Queue

This is typically done **once** during application setup or deployment, not in your regular application code.

5.3.4.1.1 REST API

Endpoint: `POST /api/v1beta/queues/durable`

The request body MAY include an optional `queue_id` field. When you omit it, the server generates a `queue_id` for you. When you supply it, the server creates the durable queue with your client-specified ID.

Queue creation is ensure-exists (idempotent): if you supply a `queue_id` that already exists, the call succeeds against the existing queue rather than returning an error. The server signals which path the request took through the HTTP status code:

- `201 Created` — the request created a new durable queue.
- `200 OK` — the request resolved to a queue that already existed; nothing was created.

Branch on the status code to tell the two apart. In both cases the response body returns the `queue_id`.

Pick an id that is unique across the whole deployment. Queue ids share a single namespace, so ensure-exists attaches you to an existing queue with your id no matter who created it or which `filter_groups` you send — you get that queue with its existing filters and `created: false`. Two callers that pick the same id share one queue: each receives the other's events, and either can delete it. Use a UUID, or a namespaced convention such as `<service-name>-<purpose>` (`thumbnailer-uploads`, say). Never use a bare generic word like `events`.

Per-queue isolation is not enforced. The three durable-queue permissions are deployment-wide rather than per-queue: a principal granted `consume-durable-queues` can consume from any durable queue whose id it knows, one granted `delete-durable-queues` can delete any of them, and `create-durable-queues` authorizes both creating a queue and updating the filters of any existing one. A `queue_id` is configuration, not a secret that protects a queue — ask your operator to grant these permissions to the service accounts that consume events rather than to broad user groups.

Server-generated `queue_id` (omit `queue_id` from the body):

```
curl -X POST "https://your-consumer-service.example.com/api/v1beta/queues/durable" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "filter_groups": [
      {
        "event_type": "storage.file.created",
        "filters": [
          {
            "filter_type": "starts_with_greedy",
            "resource_id": "/uploads/"
          }
        ]
      }
    ]
  }'
```

Client-specified `queue_id` (include `queue_id` in the body):

```
curl -i -X POST "https://your-consumer-service.example.com/api/v1beta/queues/durable"
↪\
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "queue_id": "thumbnail-service",
    "filter_groups": [
      {
        "event_type": "storage.file.created",
        "filters": [
          {
            "filter_type": "starts_with_greedy",
            "resource_id": "/uploads/"
          }
        ]
      }
    ]
  }'
```

The `-i` flag prints the status line so you can see whether the queue was created (`201 Created`) or already existed (`200 OK`). A second identical call with the same `queue_id` succeeds and returns `200 OK`.

Response:

```
{
  "queue_id": "durable-queue-abc123xyz",
  "created": true
}
```

Important: Save the `queue_id` **and** the `filter_groups` you created it with, together as one piece

of configuration. You need the `queue_id` to consume and delete, and you need both to recreate the queue if it is auto-deleted after going unconsumed for the deployment's idle window — its subscriptions are rebuilt from the filters. See *Durable Queue Lifetime*.

5.3.4.1.2 gRPC API

RPC: `CreateDurableQueue`

```python
import grpc
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

# Setup channel and stub
channel = grpc.secure_channel(
    'your-consumer-service.example.com:50052',
    grpc.ssl_channel_credentials()
)
stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)

# Define filters
filter_group = event_consumer_pb2.FilterGroup(
    event_type='storage.file.created'
)
resource_filter = event_consumer_pb2.ResourceFilter(
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
)
filter_group.filters.append(resource_filter)

# Create durable queue
request = event_consumer_pb2.CreateDurableQueueRequest()
request.filter_groups.append(filter_group)
# Optionally supply a client-specified queue_id. Omit it to have the server
# generate one. Creation is ensure-exists (idempotent): supplying a queue_id
# that already exists succeeds against the existing queue instead of erroring —
# whoever created it, and whatever filter_groups this call carries. The id must
# therefore be unique across the deployment: use a UUID or a namespaced
# <service-name>-<purpose> form, never a bare generic word.
request.queue_id = 'thumbnail-service'
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

response = stub.CreateDurableQueue(request, metadata=metadata)
queue_id = response.queue_id

# The created field mirrors the REST 201/200 distinction: it is True when this
# call created a new queue and False when an existing queue was returned unchanged.
if response.created:
    print(f"Created durable queue: {queue_id}")
else:
    print(f"Durable queue already existed: {queue_id}")
# IMPORTANT: Save this queue_id AND the filter_groups you created it with to
# your application config — you need both to recreate the queue if it is
# auto-deleted after going unconsumed for the idle window. See "Durable Queue
# Lifetime" (#durable-queue-lifetime).
```

5.3.4.2 Step 2: Consume from Durable Queue

Now consume events using the queue_id. Your application can disconnect and reconnect using the same queue_id.

Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client. Pick ids as described under *Step 1*.

5.3.4.2.1 REST API

Endpoint: `GET /api/v1beta/events/stream-durable`

```
curl -N GET "https://your-consumer-service.example.com/api/v1beta/events/stream-
↪durable?queue_id=durable-queue-abc123xyz" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

Python Example:

Messages are acknowledged to the broker automatically by the server after a short delay. Clients do not need to acknowledge individual messages, and there is no way to ask for a message to be delivered again on this endpoint. If your application requires explicit client-side acknowledgment (e.g. to guarantee at-least-once processing), or needs to hand a message it failed to process back for another attempt, use the gRPC `ConsumeDurableEventsWithAcks` RPC instead.

```
import sseclient
import requests
import json

url = "https://your-consumer-service.example.com/api/v1beta/events/stream-durable"
headers = {
    "Authorization": f"Bearer {YOUR_TOKEN}",
    "Accept": "text/event-stream"
}
params = {"queue_id": queue_id}

response = requests.get(url, headers=headers, params=params, stream=True)
client = sseclient.SSEClient(response)

for event in client.events():
    if event.event == "event":
        data = json.loads(event.data)
        process_event(data)
```

Reconnecting: Just use the same queue_id — no reconnect token needed:

```
# After disconnection or application restart
response = requests.get(url, headers=headers, params=params, stream=True)
# Will continue from where you left off
```

5.3.4.2.2 gRPC API

There are two gRPC RPCs for consuming from durable queues:

- **ConsumeDurableEventsWithAcks** (recommended) — bidirectional streaming with explicit client acknowledgment. You settle each message yourself: acknowledge it once you have processed it, or negatively acknowledge (nack) it to have it delivered again. A message you neither acknowledge nor nack is redelivered when you reconnect. Redelivery is bounded, so this is at-least-once delivery up to a retry budget: a message that exhausts that budget (or the queue's configured message TTL or maximum length) is moved to a dead-letter queue rather than redelivered indefinitely. See *Bounded Redelivery and the Dead-Letter Queue* below.
- **ConsumeDurableEvents** (deprecated) — server streaming with automatic server-side acknowledgment after a short delay. Has a window for message loss on disconnect.

Using ConsumeDurableEventsWithAcks (recommended):

```
import grpc
import queue
import threading
import time
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

# Setup channel and stub
channel = grpc.secure_channel(
    'your-consumer-service.example.com:50052',
    grpc.ssl_channel_credentials()
)
stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]


class TransientProcessingError(Exception):
    """Raised by your handler when the failure is worth another attempt."""


work_queue = queue.Queue()    # (delivery_tag, events) handed off by the stream
settle_queue = queue.Queue()  # (ack_tags, nack_tags) sent back to the service

RETRY_DELAY_SECONDS = 5  # grow this on repeated failures; stay inside the budget


def worker():
    """Process events and settle them — off the response loop."""
    while True:
        item = work_queue.get()
        if item is None:
            break
        delivery_tag, events = item
        try:
            for event in events:
                process_event(event)  # your handler; may raise
        except TransientProcessingError:
            # Wait BEFORE handing the message back: redelivery is immediate,
            # so nacking without a delay retries at full speed. The wait runs
            # here, on the worker, and never on the response loop.
            time.sleep(RETRY_DELAY_SECONDS)
```

(continues on next page)

(continued from previous page)

```
            settle_queue.put(([], [delivery_tag]))
        else:
            settle_queue.put(([delivery_tag], []))


def request_stream(queue_id):
    # First request: identify the durable queue
    yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
        queue_id=queue_id
    )
    # Subsequent requests: settle previously delivered messages
    while True:
        settlement = settle_queue.get()
        if settlement is None:
            break
        ack_tags, nack_tags = settlement
        yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
            ack_delivery_tags=ack_tags,
            nack_delivery_tags=nack_tags,
        )


threading.Thread(target=worker, daemon=True).start()

try:
    responses = stub.ConsumeDurableEventsWithAcks(
        request_stream(queue_id), metadata=metadata
    )

    # The response loop only hands work off; it never processes and never waits.
    for response in responses:
        work_queue.put((response.delivery_tag, list(response.events)))

except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.UNAUTHENTICATED:
        # Refresh token and reconnect with same queue_id
        print("Token expired, reconnecting...")
```

The retry delay is the client's responsibility, and it belongs on the worker: the service redelivers a nacked message immediately, and a `time.sleep` on the response loop would stall every other delivery on the stream while one message waits. See *Don't Block the Event Stream*.

Asking for a message to be delivered again

Send the message's `delivery_tag` in `nack_delivery_tags` instead of `ack_delivery_tags` when your handler cannot process it right now — a downstream service is unavailable, a dependency is rate-limiting you, a resource is temporarily locked. The message goes back on the queue and is delivered again.

Three properties to design around:

- **The retry is immediate**, and it may arrive on the stream you nacked from — often within milliseconds. The service applies no delay of its own, so **back off before you nack**. Without a delay you are retrying in a tight loop.
- **Nacking always returns the message to the queue.** There is no "reject and drop" — acknowledging is how you drop a message you do not want.
- **Each nack spends part of a fixed retry budget** that is shared with every other kind of redeliv-

ery. See *Bounded Redelivery and the Dead-Letter Queue* below; it is the most important thing to understand before you nack in production.

Sending the same tag in both `ack_delivery_tags` and `nack_delivery_tags` on one request acknowledges it — the acknowledgment wins. Sending a tag the service does not recognize (one you already settled, or one from a previous connection) is skipped and leaves your stream running.

Using ConsumeDurableEvents (deprecated):

```
# Consume from durable queue (auto-ack, deprecated)
request = event_consumer_pb2.ConsumeDurableEventsRequest(
    queue_id=queue_id  # Use the saved queue_id
)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

try:
    responses = stub.ConsumeDurableEvents(request, metadata=metadata)

    for response in responses:
        for event in response.events:
            print(f"Event type: {event.event_type}")
            print(f"Message: {event.message}")
            # Process event

except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.UNAUTHENTICATED:
        # Refresh token and reconnect with same queue_id
        print("Token expired, reconnecting...")
```

5.3.4.3 Durable Queue Lifetime

A durable queue ends in one of two ways: you delete it with `DeleteDurableQueue`, or the service deletes it after it has gone without an attached consumer for the deployment's **idle window** — 24 hours by default, configurable by the operator. When a queue is deleted this way, any messages it still holds are discarded; they are not moved to the dead-letter queue.

This is consistent with the message TTL that already applies to durable queues: an absent consumer's messages age out after the same window anyway, so what the idle window removes is the empty queue itself.

What keeps a queue alive

Only an **attached consumer** does. The idle clock starts when your consumer disconnects or cancels.

Action	Keeps the queue alive?
Consuming (`ConsumeDurableEvents` / `ConsumeDurableEventsWithAcks`)	**Yes** — for as long as the stream is attached
Events being published to the queue	No — a queue nobody consumes from is idle even while filling up
Calling `CreateDurableQueue` again on an existing queue	No — re-declaring does not renew the lease
Deleting and recreating the queue	Yes, but you lose the buffered messages

A client that consumes continuously, or that reconnects within the window, is never affected.

Recovering from an auto-deleted queue

Consuming from a queue that was deleted fails loudly rather than returning an empty stream — `404` on REST, `NOT_FOUND` on gRPC. Recover by recreating the queue with the same ID and resuming:

```
try:
    for response in stub.ConsumeDurableEvents(request, metadata=metadata):
        handle(response)
except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.NOT_FOUND:
        # The queue was deleted after going unconsumed for the idle window.
        # Recreate it with the SAME id and the SAME filters, then resume.
        stub.CreateDurableQueue(
            CreateDurableQueueRequest(
                queue_id=saved_queue_id,
                filter_groups=saved_filter_groups,
            ),
            metadata=metadata,
        )
        # Retry the consume with saved_queue_id.
```

Because `CreateDurableQueue` is ensure-exists, the same call is safe whether or not the queue still exists — it recreates a missing queue and is a no-op against a surviving one. Events published while the queue was gone are not recoverable.

Persist your filters, not just your queue ID

The queue's event subscriptions are rebuilt from `filter_groups` when the queue is created. An application that stored only its `queue_id` cannot fully recover a deleted queue — it would recreate the queue with whatever filters happen to be in the code path at that moment. Store the `queue_id` and its `filter_groups` together as one piece of configuration.

If your consumer is idle by design

A consumer that legitimately runs on a schedule longer than the idle window — a weekly batch job, say — should either ask the operator to raise the idle window for that deployment, or simply follow the recreate-on-`NOT_FOUND` path above at the start of each run.

5.3.4.4 Bounded Redelivery and the Dead-Letter Queue

When you consume from a durable queue with `ConsumeDurableEventsWithAcks`, a message you do not acknowledge comes back to be processed again. There are three ways that happens: you nack it, your connection drops before you settle it, or the broker takes your stream's consumer away and returns your unsettled messages to the queue.

Redelivery is bounded. Each message carries a **delivery count**, and the service delivers a message only until that count passes the deployment's configured limit. This gives you at-least-once delivery up to a retry budget, not redelivery forever.

> **The retry budget belongs to the message, not to the attempt.** The delivery count is cumulative over the message's entire life and is never reset — not by reconnecting, not by a successful delivery that you later nack, not by starting a new stream. Nacks, disconnects, and broker-side requeues all draw down the same budget. A nack is redelivered immediately, so a handler that nacks in a tight loop while a downstream service is down can spend a message's whole budget in **under a second** and dead-letter work that was only briefly unprocessable. Always back off between attempts.

A message leaves the durable queue and is moved to a dead-letter queue (DLQ) when any of the following happens:

- Its delivery count passes the configured limit (for example, a consistently failing "poison" message that is never successfully acknowledged).
- It outlives the queue's configured message TTL.
- It is evicted when the queue reaches its configured maximum length.

The practical implications for a consumer are:

- **A poison message will not block the stream indefinitely.** A message your handler can never process is redelivered only a finite number of times and is then dead-lettered, so it stops being redelivered and the rest of the stream continues to flow.
- **At-least-once is qualified.** Settle each message promptly. A message that is never successfully acknowledged within its budget is dead-lettered rather than redelivered again, so design your handlers to make forward progress within it.
- **Nack deliberately, and with backoff.** Nacking is the right response to a *transient* failure, and the wrong response to a permanent one — a message your handler will never accept is better acknowledged and recorded on your side than retried until the deployment dead-letters it. When you do nack, delay first: the budget is small, and retrying at full speed is how you exhaust it.
- **The durable queue is bounded.** A slow or stalled consumer cannot grow the queue without limit — once the queue reaches its configured maximum length, the oldest overflow messages are dead-lettered rather than retained indefinitely.

The concrete redelivery limit, message TTL, and maximum-length values are configured by the deployment and are not surfaced to consumers through the consuming RPCs; ask your operator for the limit in force if you are tuning a retry strategy against it. Operators with the appropriate permission can inspect, replay, discard, or leave dead-lettered messages through a dedicated management RPC — see *Managing the Dead-Letter Queue (Operators)* below.

5.3.4.5 Managing the Dead-Letter Queue (Operators)

The sections above describe how messages *arrive* in the dead-letter queue (DLQ). This section describes how an operator inspects and triages them.

Who this is for: This is an operator-facing capability, not a per-consumer one. It is gated by the `event-consumer-service:inspect-and-replay-all-dlq` permission. A caller that lacks this permission is denied with `PERMISSION_DENIED`.

gRPC-only: Dead-letter management is exposed only over gRPC via the `ManageDeadLetteredEvents` RPC. There is no REST/SSE equivalent.

5.3.4.5.1 The browse-and-act model

`ManageDeadLetteredEvents` is a bidirectional streaming RPC. The session works as a live browse-and-act loop:

1. The client opens the stream with an initial (empty) request.
2. The server streams dead-lettered messages to the client, one per `dead_lettered_event` response, each carrying triage metadata.
3. The client replies with per-message actions — `ACTION_REPLAY`, `ACTION_DISCARD`, or `ACTION_LEAVE` — keyed by the message's session-scoped `delivery_tag`.

4. For each action the operator sends, the server streams back an `action_outcome` response reporting what happened, correlated by the same `delivery_tag`.

The dead-letter queue is global rather than scoped to a single durable queue, so a single management session surfaces dead-lettered messages from across the deployment.

No datastore required. This bidirectional browse-and-act stream is the permanent baseline DLQ-management surface. It works against the message broker alone and requires no additional datastore. It is the forward-compatible floor for dead-letter management: future additive unary store-backed RPCs (for example, listing dead-lettered events with a server-side cursor, filtering them, replaying a specific event by id, or reporting aggregate statistics) may be added later without changing this RPC. Those additive capabilities are not available today — do not depend on them.

5.3.4.5.2 Triage metadata

Each `dead_lettered_event` carries metadata to support triage:

- **`reason`** — why the message was dead-lettered: `REASON_DELIVERY_LIMIT` (exceeded its redelivery limit), `REASON_EXPIRED` (outlived the queue's configured message TTL), or `REASON_MAXLEN` (evicted when the queue reached its configured maximum length).
- **`origin_queue`** — the name of the durable queue the message was dead-lettered from.
- **`delivery_count`** — the number of times the broker attempted delivery before dead-lettering.
- **`dead_lettered_at`** — the timestamp when the message was dead-lettered.
- **`event`** — the dead-lettered event itself, carrying its `event_type`, `principal_identity`, and `message` payload.
- **`resource_id`** — the resource id associated with the dead-lettered event.
- **`delivery_tag`** — the session-scoped tag used to act on this message (see the caveat below).

5.3.4.5.3 Actions: ACTION_REPLAY, ACTION_DISCARD, ACTION_LEAVE

The operator replies with an action per `delivery_tag`:

- **`ACTION_REPLAY`** — republishes the message to its origin queue. Replay is confirm-gated and no-loss: the dead-letter copy is removed only after the publish to the origin queue is confirmed. A replay that cannot be confirmed never drops the message — the dead-letter copy is left in place and the server reports an `OUTCOME_REPLAY_FAILED` outcome.
- **`ACTION_DISCARD`** — acknowledges the message and drops it from the dead-letter queue.
- **`ACTION_LEAVE`** — leaves the message in place in the dead-letter queue for later triage.

5.3.4.5.4 Action-outcome reports

The server-to-client stream carries two kinds of message, modeled as a response `oneof payload` with two arms: a `dead_lettered_event` (a browse item, described above) and an `action_outcome` (a report). Every action the operator sends produces exactly one correlated `action_outcome`, keyed by `delivery_tag`, so the operator can observe the result of every action rather than relying on server-side logs.

An `action_outcome` carries:

- **`delivery_tag`** — the session-scoped tag identifying which dead-lettered message the outcome pertains to (matches the tag the operator sent).
- **`outcome`** — one of `OUTCOME_REPLAYED`, `OUTCOME_DISCARDED`, `OUTCOME_LEFT`, `OUTCOME_ORIGIN_QUEUE_DELETED`, or `OUTCOME_REPLAY_FAILED`.
- **`detail`** — a human-readable note with additional context (for example, the name of the missing origin queue for an `OUTCOME_ORIGIN_QUEUE_DELETED` outcome).

The `outcome` values are:

- **`OUTCOME_REPLAYED`** — the message was successfully republished to its origin queue and the dead-letter copy was removed after the publish confirm.
- **`OUTCOME_DISCARDED`** — the message was acknowledged and dropped from the dead-letter queue.
- **`OUTCOME_LEFT`** — the message was left in place in the dead-letter queue for later triage.
- **`OUTCOME_ORIGIN_QUEUE_DELETED`** — an `ACTION_REPLAY` could not proceed because the origin queue no longer exists (see below). This outcome is no-loss.
- **`OUTCOME_REPLAY_FAILED`** — an `ACTION_REPLAY` could not be confirmed. This outcome is no-loss: the dead-letter copy is left in place exactly as for an `ACTION_LEAVE`, so the message is never dropped on a failed replay.

Origin-queue-deleted handling. A `ACTION_REPLAY` of a message whose origin queue has been deleted does **not** lose the message. The replay cannot proceed because there is no viable target, so the server leaves the dead-letter copy in place (exactly the `ACTION_LEAVE` semantics) and reports an `OUTCOME_ORIGIN_QUEUE_DELETED` outcome on the stream — the condition is reported to the operator, not silently swallowed. To obtain a viable replay target, re-create the origin queue and replay the message again.

Scope. `ACTION_REPLAY` returns a message to the exact queue it dead-lettered from. The target comes from the message's own broker metadata rather than from the request, so a replay cannot be redirected to another queue. The `event-consumer-service:inspect-and-replay-all-dlq` permission is coarse: it grants deployment-wide dead-letter visibility and action by design; finer-grained scoping is deferred. When a replayed message is subsequently re-consumed, it still runs the existing per-event authorization check.

Session delivery tags are live-only. A `delivery_tag` is valid only within the live stream that produced it. Tags are not persisted and must not be reused across reconnects. If the stream drops, re-open a new management session; the server re-presents the still-dead-lettered messages with fresh tags.

5.3.4.5.5 gRPC Python Example

```python
import grpc
import queue
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

# Setup channel and stub
channel = grpc.secure_channel(
    'your-consumer-service.example.com:50052',
    grpc.ssl_channel_credentials()
)
stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)
# The caller's token must carry the
# event-consumer-service:inspect-and-replay-all-dlq permission.
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

action_queue = queue.Queue()

def request_stream():
    # First request opens the stream and carries no actions.
    yield event_consumer_pb2.ManageDeadLetteredEventsRequest()
    # Subsequent requests carry per-delivery-tag actions.
    while True:
        entries = action_queue.get()
        if entries is None:
            break
        request = event_consumer_pb2.ManageDeadLetteredEventsRequest()
        request.actions.extend(entries)
        yield request

responses = stub.ManageDeadLetteredEvents(request_stream(), metadata=metadata)

Request = event_consumer_pb2.ManageDeadLetteredEventsRequest
Response = event_consumer_pb2.ManageDeadLetteredEventsResponse

for response in responses:
    # The response oneof has two arms: dead_lettered_event and action_outcome.
    arm = response.WhichOneof('payload')

    if arm == 'dead_lettered_event':
        dle = response.dead_lettered_event
        print(f"Dead-lettered: tag={dle.delivery_tag} "
              f"reason={Response.DeadLetteredEvent.Reason.Name(dle.reason)} "
              f"origin_queue={dle.origin_queue} "
              f"event_type={dle.event.event_type} "
              f"resource_id={dle.resource_id}")

        # Decide what to do and reply, keyed by the session delivery tag.
        entry = Request.ActionEntry(
            delivery_tag=dle.delivery_tag,
            action=Request.ACTION_REPLAY,
        )
        action_queue.put([entry])

    elif arm == 'action_outcome':
        outcome = response.action_outcome
```

(continues on next page)

(continued from previous page)

```
        name = Response.ActionOutcome.Outcome.Name(outcome.outcome)
        print(f"Outcome for tag={outcome.delivery_tag}: {name} "
              f"({outcome.detail})")

        if outcome.outcome == Response.ActionOutcome.Outcome.OUTCOME_ORIGIN_QUEUE_
↪DELETED:
            # The message was NOT lost: the dead-letter copy is left in place.
            # Re-create the origin queue, then replay again.
            print("Origin queue is gone; re-create it before replaying.")
```

5.3.4.6 Step 3: Delete Durable Queue

When your application no longer needs the queue (e.g., during decommissioning), delete it to free resources.

Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client. Pick ids as described under *Step 1*.

5.3.4.6.1 REST API

Endpoint: `DELETE /api/v1beta/queues/durable`

```
curl -X DELETE "https://your-consumer-service.example.com/api/v1beta/queues/durable?
↪queue_id=durable-queue-abc123xyz" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

Response:

```
{
  "success": true
}
```

5.3.4.6.2 gRPC API

RPC: `DeleteDurableQueue`

```
request = event_consumer_pb2.DeleteDurableQueueRequest(
    queue_id=queue_id
)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

response = stub.DeleteDurableQueue(request, metadata=metadata)
print("Queue deleted successfully")
```

5.3.4.7 Updating Durable Queue Filters

You can update filters on an existing durable queue without deleting it.

Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client. Pick ids as described under *Step 1*.

5.3.4.7.1 REST API

Endpoint: `PATCH /api/v1beta/queues/durable`

```
curl -X PATCH "https://your-consumer-service.example.com/api/v1beta/queues/durable" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "queue_id": "durable-queue-abc123xyz",
    "current_filter_groups": [
      {
        "event_type": "storage.file.created",
        "filters": []
      }
    ],
    "new_filter_groups": [
      {
        "event_type": "storage.file.created",
        "filters": []
      },
      {
        "event_type": "storage.file.deleted",
        "filters": []
      }
    ]
  }'
```

5.3.4.7.2 gRPC API

RPC: `UpdateDurableQueue`

```
request = event_consumer_pb2.UpdateDurableQueueRequest(
    queue_id=queue_id
)
for fg in current_filters:
    request.current_filter_groups.append(fg)
for fg in new_filters:
    request.new_filter_groups.append(fg)
metadata = [('authorization', f'Bearer {YOUR_TOKEN}')]

response = stub.UpdateDurableQueue(request, metadata=metadata)
```

5.3.5. Resource Filtering

5.3.5.1 Filter Types

Three types of resource filters are available:

5.3.5.1.1 1. EQ (Exact Match)

Matches only exact resource_id:

```
{
    "filter_type": "eq",
    "resource_id": "/projects/project1/file.txt"
}

# Matches: /projects/project1/file.txt
# No match: /projects/project1/file2.txt
# No match: /projects/project1/sub/file.txt
```

5.3.5.1.2 2. STARTS_WITH_LAZY (Shallow Prefix Match)

Matches resources at the same level or one level deep:

```
{
        "filter_type": "starts_with_lazy",
    "resource_id": "/projects/project1/"
}

# Matches: /projects/project1/file.txt
# No match: /projects/project1/subfolder/file.txt (too deep)
# No match: /projects/project10/file.txt (not a prefix match)
```

5.3.5.1.3 3. STARTS_WITH_GREEDY (Deep Prefix Match)

Matches resources at any depth under the prefix:

```
{
            "filter_type": "starts_with_greedy",
    "resource_id": "/projects/project1/"
}

# Matches: /projects/project1/file.txt
# Matches: /projects/project1/subfolder/file.txt
# Matches: /projects/project1/a/b/c/d/file.txt
# No match: /projects/project10/file.txt
```

5.3.5.2 Empty Resource ID

Special behavior for `resource_id=""`:

```
# STARTS_WITH_GREEDY with empty resource_id: receive ALL events of this type
{
            "filter_type": "starts_with_greedy",
    "resource_id": ""
}

# STARTS_WITH_LAZY with empty resource_id: receive events with no resource
# or single-level resources
{
        "filter_type": "starts_with_lazy",
    "resource_id": ""
}

# EQ with empty resource_id: ERROR (not allowed)
{
    "filter_type": "eq",
    "resource_id": ""
}
```

5.3.5.3 Multiple Filters (OR Logic)

Multiple filters within a FilterGroup use OR logic:

```
# Receive events from /uploads/ OR /shared/
{
    "event_type": "storage.file.created",
    "filters": [
        {
            "filter_type": "starts_with_greedy",
            "resource_id": "/uploads/"
        },
        {
            "filter_type": "starts_with_greedy",
            "resource_id": "/shared/"
        }
    ]
}
```

5.3.5.4 Multiple Event Types

Multiple FilterGroups let you subscribe to multiple event types:

```
[
    {
        "event_type": "storage.file.created",
        "filters": [
            {"filter_type": "starts_with_greedy", "resource_id": "/uploads/"}
        ]
    },
    {
```

(continues on next page)

(continued from previous page)

```
        "event_type": "storage.file.deleted",
        "filters": [
            {"filter_type": "starts_with_greedy", "resource_id": "/uploads/"}
        ]
    }
]
```

5.3.6. Error Handling

5.3.6.1 Authorization Errors During Streaming

Important: Authorization errors can occur **at any time** during streaming, not just at connection time.

For example:

- Your authentication token expires while streaming
- Permissions change while you're connected

When this happens, the stream will terminate with a 401 (UNAUTHENTICATED) or 403 (PERMISSION_DENIED) error.

Handling in REST/SSE:

```
import time

def consume_with_auto_reconnect(url, get_token_func, queue_id=None):
    """
    Consume events with automatic reconnection on auth errors.
    Messages are acknowledged automatically by the server.
    """
    while True:
        try:
            token = get_token_func()  # Get fresh token
            headers = {
                "Authorization": f"Bearer {token}",
                "Accept": "text/event-stream"
            }

            params = {}
            if queue_id:
                params["queue_id"] = queue_id

            response = requests.get(url, headers=headers, params=params, stream=True)

            if response.status_code == 401 or response.status_code == 403:
                print("Auth error, refreshing token...")
                time.sleep(1)
                continue

            client = sseclient.SSEClient(response)

            for event in client.events():
                if event.event == "event":
                    data = json.loads(event.data)
```

(continues on next page)

(continued from previous page)

```
                    process_event(data)

        except requests.exceptions.RequestException as e:
            print(f"Connection error: {e}, reconnecting...")
            time.sleep(5)
```

Handling in gRPC:

```
import queue

def consume_with_retry(stub, queue_id, get_token_func):
    """
    Consume with automatic reconnection on auth errors.
    Uses ConsumeDurableEventsWithAcks for explicit acknowledgment.
    """
    while True:
        try:
            token = get_token_func()
            metadata = [('authorization', f'Bearer {token}')]
            ack_queue = queue.Queue()

            def request_stream():
                yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
                    queue_id=queue_id
                )
                while True:
                    tags = ack_queue.get()
                    if tags is None:
                        break
                    yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
                        ack_delivery_tags=tags
                    )

            responses = stub.ConsumeDurableEventsWithAcks(
                request_stream(), metadata=metadata
            )

            for response in responses:
                for event in response.events:
                    process_event(event)
                ack_queue.put([response.delivery_tag])

        except grpc.RpcError as e:
            if e.code() in [grpc.StatusCode.UNAUTHENTICATED,
                            grpc.StatusCode.PERMISSION_DENIED]:
                print("Auth error, refreshing token and reconnecting...")
                time.sleep(1)
            elif e.code() == grpc.StatusCode.RESOURCE_EXHAUSTED:
                print("Service overloaded, backing off...")
                time.sleep(5)
            elif e.code() == grpc.StatusCode.UNAVAILABLE:
                print("Connection lost, reconnecting...")
                time.sleep(1)
            else:
                print(f"Error: {e.code()}: {e.details()}")
                time.sleep(5)
```

5.3.6.2 Other Common Errors

HTTP Status / gRPC Code	Meaning	Action
400 / INVALID_ARGUMENT	Invalid filter_groups	Check your request format
422 / INVALID_ARGUMENT	A `queue_id` that does not match `^[A-Za-z0-9._-]{1,200}$`, or a reconnect token that is not 1 to 200 printable ASCII characters (no spaces, control characters, or non-ASCII), whether it arrives as the `reconnect_token` query parameter, in the `Last-Event-ID` header, or in the gRPC request field. Applies on every operation that accepts one, not just creation	Correct the value — the request is rejected before it reaches the broker. Send a reconnect token back exactly as the server issued it
404 / NOT_FOUND	Queue not found (durable) — either the queue_id is wrong, or the queue was auto-deleted after going unconsumed for the idle window	Verify queue_id, then recreate the queue with the same id and filters — see *Durable Queue Lifetime*
500 / INTERNAL — `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`	The broker cancelled this stream's consumer, or killed the channel it lived on — commonly an acknowledgement timeout on a slow consumer, or a broker-side leader change. Applies to every consume stream: durable, non-durable, and dead-letter management	Reconnect and resume consuming; do **not** recreate the queue. The queue normally survives with its buffered messages, and messages that were in flight unacknowledged are redelivered after you reconnect. This error does not by itself confirm that the queue survived: if the queue no longer exists, the reconnect returns `NOT_FOUND` / `404`, and the recovery for a queue that is gone applies — see *Handle Disconnections Gracefully*
503 / RESOURCE_EXHAUSTED	Service temporarily overloaded	Retry with exponential backoff
— / UNAVAILABLE	Connection lost (e.g. load balancer idle timeout)	Reconnect automatically

5.3.7. Best Practices

5.3.7.1 1. Choose the Right Queue Type

See the *decision matrix* above.

5.3.7.2 2. Handle Disconnections Gracefully

```
# Non-durable: Save reconnect_token
reconnect_token = None
for response in responses:
    if response.reconnect_token:
        reconnect_token = response.reconnect_token

# Use it to reconnect
request = ConsumeNonDurableEventsRequest(reconnect_token=reconnect_token)

# Durable: Just use the same queue_id
request = ConsumeDurableEventsWithAcksRequest(queue_id=saved_queue_id)
```

A non-durable reconnect resumes the queue with the bindings it already holds, so the token alone is enough only while your filter set still matches the last set the server acknowledged. A gRPC client that changes filters mid-stream tracks which of its updates were acknowledged and, after reconnecting with the token, re-sends in order each update it sent but did not see acknowledged, each naming in `previous_filter_groups` the set it replaced — see *Updating Filters Dynamically*. If the server does not set `supports_filter_update_acks`, open a fresh stream with `filter_groups` and no `reconnect_token` instead of resuming.

A stream also ends when the broker drops your consumer. Every consume stream — durable and non-durable, over gRPC (with or without client acknowledgment) and over REST/SSE alike, and the dead-letter management stream — is attached to its queue by a consumer on the broker. The broker can take that consumer away underneath an otherwise healthy connection: when the queue is deleted, and when it kills the channel the consumer lives on (on durable queues, an acknowledgement timeout on a slow consumer is the usual cause). The stream ends with an error when this happens, so handle it: a client that ignores the error and keeps waiting on the stream receives no further events. The check runs while the stream is idle, so the error can arrive up to about 15 seconds after the loss on a default deployment.

Two errors are distinguished, because they call for opposite recoveries. How each one reaches you depends on the protocol: on gRPC the stream terminates with the status code shown; on REST/SSE the HTTP response status was already `200 OK` when the stream opened, so the same condition arrives as an SSE event named `error` whose `data` carries the `message` and the matching `status_code`, and the stream then closes. An SSE client must therefore watch for an event named `error`, not only for a broken connection.

Error	Meaning	Action
`500 / INTERNALInternal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`	The consumer attached to the queue was lost; the queue normally survives	**Reconnect — do not recreate the queue.** Consume again with the same `queue_id` (durable) or `reconnect_token` (non-durable). If that reconnect returns `404 / NOT_FOUND`, follow the row below.
`404 / NOT_FOUND`	The queue itself no longer exists	**Recreate, then resume.** Durable: call `CreateDurableQueue` with the same `queue_id` **and** `filter_groups`. Non-durable: see below

Reconnecting after `INTERNAL` costs you nothing: the queue normally survives with everything buffered in it, and **messages that were in flight unacknowledged are redelivered once you reconnect**. `INTERNAL` does not by itself confirm that the queue survived: if it no longer exists, the reconnect returns `NOT_FOUND`, and the row above applies. Process events idempotently so those redeliveries are safe — see *Process Events Idempotently*.

`NOT_FOUND` has one extra nuance for non-durable consumers. A non-durable queue outlives its consumer only for the deployment's brief reconnect-grace window (20 seconds by default). Within that window your `reconnect_token` still resolves and you resume with the buffered backlog intact. Once the window lapses the queue is gone and reconnecting with the token returns `NOT_FOUND` again — **start a fresh stream with your `filter_groups` and no `reconnect_token`**, which creates a new queue. Events published while the queue was gone are not recoverable.

```python
try:
    for response in responses:
        handle(response)
except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.INTERNAL:
        # The consumer was lost; the queue normally survives — reconnect.
        # If the reconnect returns NOT_FOUND, take the NOT_FOUND branch below.
        # Unacknowledged messages are redelivered.
        reconnect()
    elif e.code() == grpc.StatusCode.NOT_FOUND:
        # The queue is gone. Durable: recreate it with the saved queue_id
        # AND filter_groups. Non-durable: start a fresh stream from
        # filter_groups, dropping the reconnect_token.
        recreate_then_reconnect()
```

The REST/SSE equivalent reads the same two outcomes off the `error` event:

```python
for event in client.events():
    if event.event == "event":
        handle(json.loads(event.data))
    elif event.event == "error":
        details = json.loads(event.data)
        if details["status_code"] == 500:
            # The consumer was lost; the queue normally survives — reconnect.
            # If the reconnected stream reports 404, take the 404 branch below.
            reconnect()
        elif details["status_code"] == 404:
```

(continues on next page)

(continued from previous page)

```
            # The queue is gone. Durable: recreate it with the saved queue_id
            # AND filter_groups. Non-durable: start a fresh stream from
            # filter_groups, dropping the reconnect_token.
            recreate_then_reconnect()
        break  # the stream closes after an error event either way
```

For durable queues, the `NOT_FOUND` path is the same one you take when a queue is auto-deleted after going without a consumer for longer than the idle window. See *Durable Queue Lifetime*.

5.3.7.3 3. Process Events Idempotently

Events may be delivered more than once — for durable queues, a message you nack or leave unsettled is redelivered until its delivery count passes the configured limit, after which it is dead-lettered. Design your event handlers to be idempotent so a redelivered message is safe to process again:

```
def process_event(event):
    event_id = get_event_id(event)

    # Check if already processed
    if redis.exists(f"processed:{event_id}"):
        print(f"Event {event_id} already processed, skipping")
        return

    # Process event
    do_work(event)

    # Mark as processed
    redis.set(f"processed:{event_id}", "1", ex=86400)  # 24h TTL
```

5.3.7.4 4. Don't Block the Event Stream

Process events quickly or hand them off to a worker:

```
import threading
import queue

event_queue = queue.Queue()

def event_processor():
    """Worker thread to process events."""
    while True:
        event = event_queue.get()
        if event is None:
            break
        process_event_slowly(event)

# Start worker
worker = threading.Thread(target=event_processor)
worker.start()

# Consume events quickly
for response in responses:
    for event in response.events:
        event_queue.put(event)  # Don't block the stream
```

5.3.7.5 5. Monitor Queue Depth (Durable Queues)

If you're not consuming fast enough, the durable queue will grow. The queue is bounded — it does not grow without limit. Once it reaches its configured maximum length, overflow messages are dead-lettered rather than retained (see *Bounded Redelivery and the Dead-Letter Queue*), so a sustained backlog can lead to dropped (dead-lettered) messages. Monitor queue depth and scale your consumers before the backlog approaches the maximum length.

5.3.7.6 6. Delete Durable Queues When Done

Always delete durable queues when your application is decommissioned:

```
# In your application shutdown handler
def cleanup():
    stub.DeleteDurableQueue(
        DeleteDurableQueueRequest(queue_id=queue_id),
        metadata=metadata
    )
```

This matters most for applications that roll their queue ID — for example, one queue per deployed generation. An abandoned queue keeps accumulating and dead-lettering events until the idle window deletes it, so deleting it yourself on rollover is still the right thing to do; the idle window is a backstop, not a substitute. See *Durable Queue Lifetime*.

5.3.7.7 7. Use Appropriate Filters

Don't receive more events than you need:

```
# Bad - receiving all events then filtering in code
bad_filter_group = FilterGroup(event_type="storage.file.created")
# No filters - receiving everything
for response in responses:
    for event in response.events:
        if event.message['file_name'].endswith('.pdf'):
            process(event)  # Wasting bandwidth

# Better - filter by resource if possible
better_filter_group = FilterGroup(
    event_type="storage.file.created"
)
resource_filter = ResourceFilter(
    filter_type=FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id="/pdfs/"  # Only get events from /pdfs/
)
better_filter_group.filters.append(resource_filter)
```

5.3.8. Complete Example: Thumbnail Generation Service

Here's a complete example of a service that generates thumbnails for uploaded images using a durable queue with explicit acknowledgment:

```
import grpc
import logging
import queue
import time
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2
from nvidia.omniverse.notifications.consumer.v1beta import event_consumer_pb2_grpc

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ThumbnailService:
    def __init__(self, consumer_endpoint, auth_token):
        self.consumer_endpoint = consumer_endpoint
        self.auth_token = auth_token
        self.queue_id = None

    def setup(self):
        """Create durable queue - call once during deployment."""
        channel = grpc.secure_channel(
            self.consumer_endpoint,
            grpc.ssl_channel_credentials()
        )
        stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)

        # Create queue for image upload events
        filter_group = event_consumer_pb2.FilterGroup(
            event_type='storage.file.created'
        )
        resource_filter = event_consumer_pb2.ResourceFilter(
            filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_
↪GREEDY,
            resource_id='/images/'
        )
        filter_group.filters.append(resource_filter)

        request = event_consumer_pb2.CreateDurableQueueRequest()
        request.filter_groups.append(filter_group)
        metadata = [('authorization', f'Bearer {self.auth_token}')]

        response = stub.CreateDurableQueue(request, metadata=metadata)
        self.queue_id = response.queue_id

        logger.info(f"Created durable queue: {self.queue_id}")
        # SAVE THIS QUEUE_ID AND ITS FILTER_GROUPS TO YOUR CONFIG — you need
        # both to recreate the queue if it is auto-deleted after going
        # unconsumed for the idle window. See "Durable Queue Lifetime"
        # (#durable-queue-lifetime).

    def _request_stream(self, ack_queue):
        """Generate requests for the bidirectional stream."""
        # First request: identify the durable queue
        yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
```

(continues on next page)

(continued from previous page)

```
            queue_id=self.queue_id
        )
        # Subsequent requests: send acknowledgments
        while True:
            tags = ack_queue.get()
            if tags is None:
                break
            yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
                ack_delivery_tags=tags
            )

    def run(self, queue_id):
        """Run the service - consumes events and generates thumbnails."""
        self.queue_id = queue_id

        while True:
            try:
                channel = grpc.secure_channel(
                    self.consumer_endpoint,
                    grpc.ssl_channel_credentials()
                )
                stub = event_consumer_pb2_grpc.EventConsumerServiceStub(channel)
                metadata = [('authorization', f'Bearer {self.auth_token}')]
                ack_queue = queue.Queue()

                logger.info("Starting to consume events...")
                responses = stub.ConsumeDurableEventsWithAcks(
                    self._request_stream(ack_queue), metadata=metadata
                )

                for response in responses:
                    for event in response.events:
                        self.process_event(event)
                    # Acknowledge after processing
                    ack_queue.put([response.delivery_tag])

            except grpc.RpcError as e:
                logger.error(f"gRPC error: {e.code()}: {e.details()}")
                if e.code() in [grpc.StatusCode.UNAUTHENTICATED,
                                grpc.StatusCode.PERMISSION_DENIED]:
                    # Refresh token and reconnect
                    self.auth_token = self.get_fresh_token()
                    logger.info("Refreshed auth token, reconnecting...")
                else:
                    logger.error("Connection failed, retrying in 5s...")
                    time.sleep(5)

    def process_event(self, event):
        """Process a single file created event."""
        try:
            file_path = event.message.get('file_path', '')
            logger.info(f"Processing file: {file_path}")

            # Generate thumbnail
            self.generate_thumbnail(file_path)
```

(continues on next page)

(continued from previous page)

```
            logger.info(f"Thumbnail generated for: {file_path}")

        except Exception as e:
            logger.error(f"Error processing event: {e}")
            # Don't raise - continue processing other events

    def generate_thumbnail(self, file_path):
        """Generate thumbnail for the given file."""
        # Implementation here
        pass

    def get_fresh_token(self):
        """Get a fresh auth token."""
        # Implementation here
        pass

# Usage
if __name__ == "__main__":
    service = ThumbnailService(
        consumer_endpoint="your-consumer-service.example.com:50052",
        auth_token="YOUR_TOKEN"
    )

    # One-time setup (or load from config)
    # service.setup()

    # Run service with saved queue_id
    service.run(queue_id="saved-queue-id-from-config")
```

5.3.9. Next Steps

- Understand *Permissions & Authorization* for consuming events
- Review the *API Reference* for complete API documentation
- Learn about *Publishing Events*

5.4. Permissions & Authorization

This guide explains how authentication and authorization work in the Notifications Service, including special cases for storage events and configurable per-event-type permissions.

5.4.1. Overview

The Notifications Service integrates with a permissions API to control:

- **Who can publish** events of specific types
- **Who can create, delete, and consume from durable queues**
- **Who can consume specific events** from non-durable queues
- **Fine-grained access control** based on event content

5.4.2. Authentication

All requests to both the Event Aggregation Service and Event Consumer Service require authentication via bearer tokens.

5.4.2.1 Including Auth Token

REST API:

```
Authorization: Bearer YOUR_JWT_TOKEN
```

gRPC:

```
metadata = [('authorization', 'Bearer YOUR_JWT_TOKEN')]
stub.PublishEvent(request, metadata=metadata)
```

5.4.2.2 Token Expiration

Important: Tokens can expire **while you're streaming events**. When this happens:

1. The stream will terminate with a 401 (UNAUTHENTICATED) error
2. Your client must refresh the token
3. Reconnect with the fresh token
   - **Non-durable queues**: Use the `reconnect_token` to avoid missing events
   - **Durable queues**: Simply reconnect with the same `queue_id`

See the *Consuming Events guide* for code examples.

5.4.3. Publishing Permissions

5.4.3.1 Event Type Permissions

To publish an event, the authenticated principal must have permission for that specific event type.

The Event Aggregation Service checks permissions using:

- **Action**: `publish-event`
- **Resource Type**: `EventType`
- **Resource ID**: The event_type string (e.g., "storage.file.created")

5.4.3.2 Example Policy

In your permissions service (e.g., Cedar policy):

```
// Allow user to publish storage events
permit(
  principal == Principal::"user@example.com",
  action == Action::"event-aggregation-service:publish-event",
  resource == EventType::"storage.file.created"
);

// Allow a service to publish workflow events
permit(
  principal == Principal::"workflow-service",
  action == Action::"event-aggregation-service:publish-event",
  resource == EventType::"workflow.completed"
);
```

5.4.3.3 REST Example

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "event": {
      "event_type": "myapp.user.created",
      ...
    }
  }'
```

If the principal doesn't have `publish-event` permission for `EventType::"myapp.user.created"`, you'll receive:

```
HTTP 403 Forbidden
{
  "detail": "Not allowed to publish events of type myapp.user.created"
}
```

5.4.3.4 Configuration

The Event Aggregation Service is configured via environment variables:

Variable	Description
`OMNI_EVENTS_PERMISSIONS_ENDPO`	URL of the permissions service. If not set, permissions checks are **disabled**.
`OMNI_EVENTS_PERMISSIONS_TTL_S`	Cache TTL for permissions checks (default: 600)

Example:

```
OMNI_EVENTS_PERMISSIONS_ENDPOINT=https://permissions.example.com
OMNI_EVENTS_PERMISSIONS_TTL_SECONDS=300
```

5.4.4. Consuming Permissions

Consuming permissions are more complex because they involve:

1. General permissions for queue management
2. Special handling for storage events
3. Optional fine-grained per-event-type permissions

5.4.4.1 Durable Queue Permissions

To work with durable queues, principals need specific permissions:

There are three of them — `create-durable-queues`, `consume-durable-queues`, and `delete-durable-queues` — covering four operations: updating a queue's filters is authorized by `create-durable-queues`, and there is no separate update action.

Each is a service-level permission, not a per-queue one: a principal granted `consume-durable-queues` may consume from any durable queue whose `queue_id` it knows, one granted `delete-durable-queues` may delete any durable queue in the deployment, and one granted `create-durable-queues` may re-filter any of them as well as create new ones. Grant them to the service accounts that operate durable queues rather than to broad user groups — see *Use Service Accounts for Durable Queues*.

5.4.4.1.1 Create Durable Queue

- **Action**: `create-durable-queues`
- **Resource**: None (service-level permission)

Policy Example:

```
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:create-durable-queues"
);
```

5.4.4.1.2 Update Durable Queue

- **Action**: `create-durable-queues`
- **Resource**: None (service-level permission)

Updating the filters on an existing durable queue is authorized by the same action as creating one, so the policy example above covers it. No `update-durable-queues` action exists.

5.4.4.1.3 Consume from Durable Queue

- **Action**: `consume-durable-queues`
- **Resource**: None (service-level permission)

Policy Example:

```
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:consume-durable-queues"
);
```

5.4.4.1.4 Delete Durable Queue

- **Action**: `delete-durable-queues`
- **Resource**: None (service-level permission)

Policy Example:

```
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:delete-durable-queues"
);
```

5.4.4.2 Durable Queue Event Permissions

When consuming from a durable queue, **additional permissions checks** are performed on individual events based on their event type.

For storage events, the service checks:

- `storage.file.created` → Action: `consume-all-storage-create-events`
- `storage.file.deleted` → Action: `consume-all-storage-delete-events`

Policy Example:

```
// Allow thumbnail service to consume all file creation events
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:consume-all-storage-create-events"
);
```

For non-storage events with configured permissions (see the Fine-Grained Permissions for Arbitrary Event Types section below), the action and resource from the configuration are used.

5.4.4.3 Non-Durable Queue Permissions

Non-durable queues don't require special queue management permissions, but **every event** is checked for permissions before being delivered.

This is where the sophisticated permission models come into play:

1. **Storage events**: Use storage-API-based "docs" permissions
2. **Configured event types**: Use permissions API with configured actions
3. **Unconfigured event types**: No permissions check (allowed by default)

5.4.5. Storage Events: Special "Docs" Permissions

Storage events have special handling because they relate to files and directories that have their own access control in the Storage API.

5.4.5.1 Storage Event Types

The following event types are treated as storage events:

- `omni.storage.created` - File created
- `omni.storage.deleted` - File deleted
- `omni.storage.dir_created` - Directory created
- `omni.storage.dir_deleted` - Directory deleted

(Note: The actual event type strings may vary based on your storage service configuration)

5.4.5.2 How Storage Permissions Work

For non-durable queues, the Event Consumer Service queries the **Storage API** to check if the user has access to the files/directories in the events.

This is sometimes called **"docs" permissions** because it's based on the actual documents/files the user can access in storage.

5.4.5.2.1 For `storage.file.created` Events

When a file is created:

1. The consumer service checks if the user has access to the **parent directory**
2. If they have any files in that directory (i.e., they can "see" the directory), they receive the event
3. The file is added to a cache so that if it's deleted later, the service knows they should receive the delete event

5.4.5.2.2 For `storage.file.deleted` Events

When a file is deleted:

1. The service checks if the user had access to that file (from the cache or by checking the parent directory)
2. If they had access, they receive the delete event
3. The file is removed from the cache

5.4.5.2.3 For `storage.dir_created` Events

When a directory is created:

1. The service checks if the user has access to the **parent directory**
2. It queries the Storage API to see if they have any files in the **new directory**
3. If they have files in the new directory, they receive the event
4. The directory is added to the cache

5.4.5.2.4 For `storage.dir_deleted` Events

When a directory is deleted:

1. The service checks if the user had access to the directory (from the cache)
2. It checks if they still have files in that "deleted" directory (in case it's a synthetic delete event)
3. If the directory was truly deleted for them, they receive the event
4. The directory is removed from the cache

5.4.5.3 Why "Docs" Permissions?

This approach ensures that:

- Users only see events for files they have access to
- A user creating a file in a shared folder only sees that event if they can access the folder
- Delete events are only sent if the user could see the file/directory in the first place
- Permissions are enforced at the storage layer, not duplicated in the events system

5.4.5.4 Configuration for Storage Events

The Event Consumer Service needs to know which Storage API endpoints to query:

Environment Variable: `OMNI_EVENTS_PERMISSIONS_YAML_FILE_PATH`

YAML Configuration File:

```
storage_permissions_endpoints:
  - https://storage-api-1.example.com
  - https://storage-api-2.example.com
```

Example:

```
OMNI_EVENTS_PERMISSIONS_YAML_FILE_PATH=/etc/event-consumer/permissions.yml
```

In the YAML file (`/etc/event-consumer/permissions.yml`):

```
storage_permissions_endpoints:
  - https://azure-blob-storage-api.example.com
  - https://s3-storage-api.example.com
```

Currently, the first endpoint in the list is used. In the future, routing rules will determine which endpoint to use based on the event's `resource_id`.

5.4.5.5 Storage Permissions with Durable Queues

Important difference: When using **durable queues**, storage events don't use the Storage API "docs" permissions.

Instead, they use simple action-based permissions:

- `storage.file.created` → `consume-all-storage-create-events`
- `storage.file.deleted` → `consume-all-storage-delete-events`

Why? Durable queues are typically used by services (not end users) that need to process all storage events, not just those for files they own.

Policy Example:

```
// Thumbnail service can consume all file creation events
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:consume-all-storage-create-events"
);

// Audit service can consume all file deletion events
permit(
  principal == Principal::"audit-service",
  action == Action::"event-consumer-service:consume-all-storage-delete-events"
);
```

5.4.6. Fine-Grained Permissions for Arbitrary Event Types

For non-storage event types, you can configure **fine-grained permissions** that check not just the event type, but also the **resource_id** from the event.

5.4.6.1 When to Use Fine-Grained Permissions

Use fine-grained permissions when:

- Different users should see different events of the same type
- Events relate to resources with their own access control (like files, projects, workspaces)
- You need to enforce permissions based on the event's `resource_id`

Examples:

- A user should only see `project.updated` events for projects they have access to
- A user should only see `thumbnail.generated` events for their own files
- A user should only see `notification.created` events for notifications sent to them

5.4.6.2 Configuration

Fine-grained permissions are configured via a YAML file specified by the `OMNI_EVENTS_PERMISSIONS_YAML_FILE_PATH` environment variable.

Environment Variable:

```
OMNI_EVENTS_PERMISSIONS_YAML_FILE_PATH=/etc/event-consumer/permissions.yml
```

YAML Configuration File:

```yaml
event_permissions:
  - event_type: project.updated
    action: read-project
  - event_type: thumbnail.generated
    action: read-thumbnail
  - event_type: notification.created
    action: read-notification
```

5.4.6.3 How It Works

For each event of a configured type:

1. The Event Consumer Service makes a permissions API call
2. It uses the configured `action`
3. It uses the event's `resource_id` as the resource ID
4. The resource type is always `EventResourceId`
5. If the principal has permission, the event is delivered; otherwise, it's filtered out

5.4.6.4 Permissions API Call

When consuming an event like:

```
{
  "event_type": "project.updated",
  "resource_id": "/projects/project-123",
  "message": {}
}
```

The service makes this permissions check:

- **Principal**: The authenticated user
- **Action**: `read-project` (from config)
- **Resource Type**: `EventResourceId`
- **Resource ID**: `/projects/project-123` (from event)

5.4.6.5 Policy Examples

Scoped Policy (using resource_id):

```
// Allow user to see events for project-123
permit(
  principal == Principal::"user@example.com",
  action == Action::"event-consumer-service:read-project",
  resource == EventResourceId::"/projects/project-123"
);
```

Global Policy (ignoring resource_id):

```
// Allow admin to see all project events
permit(
  principal == Principal::"admin@example.com",
  action == Action::"event-consumer-service:read-project"
);
```

Hierarchical Policy:

```
// Allow user to see events for any project they own
permit(
  principal == Principal::"user@example.com",
  action == Action::"event-consumer-service:read-project",
  resource == EventResourceId
)
when {
  resource in Principal::"user@example.com"
};
```

5.4.6.6 Complete Configuration Example

YAML file (`/etc/event-consumer/permissions.yml`):

```yaml
# Storage API endpoints for "docs" permissions
storage_permissions_endpoints:
  - https://storage-api.example.com

# Fine-grained event permissions
event_permissions:
  # Project events
  - event_type: project.created
    action: read-project
  - event_type: project.updated
    action: read-project
  - event_type: project.deleted
    action: read-project

  # Thumbnail events
  - event_type: thumbnail.generated
    action: read-thumbnail

  # Notification events
  - event_type: notification.created
    action: read-notification

  # Workflow events
  - event_type: workflow.completed
    action: read-workflow
```

Corresponding Policies:

```
// User can see their own project events
permit(
  principal == Principal::"user@example.com",
  action in [
    Action::"event-consumer-service:read-project"
  ],
  resource == EventResourceId
)
when {
  // Check if resource (project) is owned by user
  resource.owner == principal.id
};

// User can see thumbnails for files they can access
permit(
  principal == Principal::"user@example.com",
  action == Action::"event-consumer-service:read-thumbnail",
  resource == EventResourceId
)
when {
  // Check via storage permissions
  resource.accessible_by(principal)
};

// All users can see notification events for their own notifications
permit(
```

(continues on next page)

(continued from previous page)

```
  principal,
  action == Action::"event-consumer-service:read-notification",
  resource == EventResourceId
)
when {
  resource.recipient == principal.id
};
```

5.4.7. Permissions Decision Flow

Here's how permissions are checked when consuming events:

5.4.7.1 Non-Durable Queues

```
Event Received



Is it a storage event?
(storage.created, storage.deleted, etc)

         Yes                  No


Check Storage API       Is event_type configured
"docs" permissions      in permissions.yml?

                               Yes            No


                      Check               Allow
                      permissions         (no check)
                      API with
                      configured
                      action +
                      resource_id



Allow or Deny?

         Allow                Deny


Deliver Event         Filter Out Event
```

5.4.7.2 Durable Queues

```
Event Received



Is it a storage event?

         Yes                  No


Check:                  Is event_type configured
- storage.created       in permissions.yml?
    consume-all-
    storage-create              Yes            No
    -events
- storage.deleted
    consume-all-        Check perms        Allow
    storage-delete      API with
    -events             action +
                        resource_id



Allow or Deny?

         Allow                Deny


Deliver Event         Filter Out Event
```

5.4.8. Summary Table

Scenario	Queue Type	Event Type	Permission Check
Create queue	Durable	N/A	`create-durable-queues`
Delete queue	Durable	N/A	`delete-durable-queues`
Consume from queue	Durable	N/A	`consume-durable-queues`
Storage create event	Durable	storage.*	`consume-all-storage-create-events`
Storage delete event	Durable	storage.*	`consume-all-storage-delete-events`
Storage event	Non-durable	storage.*	Storage API “docs” permissions
Configured event	Either	Any	Configured action + EventResourceId
Unconfigured event	Either	Any	No check (allowed)
Publishing	N/A	Any	`publish-event` with EventType

5.4.9. Configuration Examples

5.4.9.1 Example 1: Thumbnail Service (Durable Queue)

The thumbnail service needs to process all file creation events.

Permissions Policies:

```
permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:create-durable-queues"
);

permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:consume-durable-queues"
);

permit(
  principal == Principal::"thumbnail-service",
  action == Action::"event-consumer-service:consume-all-storage-create-events"
);
```

No YAML configuration needed - the service uses simple action-based permissions.

5.4.9.2 Example 2: User Dashboard (Non-Durable Queue)

A web dashboard shows users file events for files they have access to.

Permissions Policies:

```
// No special policies needed - users use their existing storage permissions
// The Event Consumer Service will query the Storage API to check access
```

YAML Configuration (`/etc/event-consumer/permissions.yml`):

```
storage_permissions_endpoints:
  - https://storage-api.example.com
```

5.4.9.3 Example 3: Project Notification Service (Non-Durable + Fine-Grained)

A service that shows users events for projects they have access to.

Permissions Policies:

```
permit(
  principal,
  action in [
    Action::"event-consumer-service:read-project"
  ],
  resource == EventResourceId
)
when {
  // User has access if they're a member of the project
```

(continues on next page)

(continued from previous page)

```
  resource.members.contains(principal.id)
};
```

YAML Configuration:

```yaml
event_permissions:
  - event_type: project.created
    action: read-project
  - event_type: project.updated
    action: read-project
  - event_type: project.deleted
    action: read-project
  - event_type: project.member.added
    action: read-project
```

5.4.10. Best Practices

5.4.10.1 1. Use Service Accounts for Durable Queues

Create dedicated service principals for services using durable queues:

```
permit(
  principal == Principal::"service:thumbnail-generator",
  action == Action::"event-consumer-service:create-durable-queues"
);
```

The durable-queue actions are deployment-wide rather than per-queue, so anyone granted them can reach any durable queue whose `queue_id` they know. Keep the grants on named service principals rather than on broad user groups, and give each service a namespaced `queue_id` (`<service-name>-<purpose>`) so queues stay attributable.

5.4.10.2 2. Document Your Permission Requirements

When defining new event types, document what permissions are needed:

```
"""
Event Type: project.updated

Publishing Permission:
  - Action: publish-event
  - Resource: EventType::"project.updated"

Consuming Permission (fine-grained):
  - Action: read-project
  - Resource: EventResourceId (project's resource_id)

Example policy:
  permit(
    principal,
    action == Action::"event-consumer-service:read-project",
    resource == EventResourceId
```

(continues on next page)

(continued from previous page)

```
  )
  when { resource.members.contains(principal.id) };
"""
```

5.4.10.3 3. Test Permissions Thoroughly

Test both positive and negative cases:

- Can authorized users publish/consume?
- Are unauthorized users properly denied?
- Do permissions work correctly after token refresh?
- Are fine-grained permissions enforced correctly?

5.4.10.4 4. Monitor Permission Denials

Log and monitor permission denials to detect:

- Misconfigured policies
- Users attempting unauthorized access
- Services missing required permissions

5.4.10.5 5. Use Hierarchical Policies Where Appropriate

For resource hierarchies, use hierarchical policies:

```
// Allow user to see events for anything in their workspace
permit(
  principal == Principal::"user@example.com",
  action == Action::"event-consumer-service:read-file",
  resource == EventResourceId
)
when {
  resource.path.startsWith("/workspaces/" + principal.workspace_id + "/")
};
```

5.4.11. Troubleshooting

5.4.11.1 "Permission denied" when publishing

Check:

1. Is `OMNI_EVENTS_PERMISSIONS_ENDPOINT` configured?
2. Does your principal have `publish-event` permission for the EventType?
3. Is your auth token valid and not expired?

5.4.11.2 "Permission denied" when creating durable queue

Check:

1. Does your principal have `create-durable-queues` permission?
2. Is your auth token valid?

5.4.11.3 Not receiving storage events (non-durable)

Check:

1. Is `storage_permissions_endpoints` configured correctly in the YAML file?
2. Does the Storage API show that the user has access to files in the relevant directories?
3. Are the event types correct? (Check the actual event type strings used by your storage service)

5.4.11.4 Not receiving events with fine-grained permissions

Check:

1. Is the event_type configured in the `event_permissions` section of the YAML file?
2. Does your policy grant the configured action for the resource_id in the events?
3. Is the `resource_id` in the events formatted as expected?

5.4.11.5 Token expiring during stream

Solution: Implement automatic token refresh and reconnection. See *Consuming Events - Error Handling*.

5.4.12. Next Steps

- Review *Publishing Events* to understand how to publish with proper authorization
- Review *Consuming Events* to understand queue types and consumption patterns
- Check the *API Reference* for complete API documentation

5.5. API Reference

This page provides detailed reference information for both the Event Aggregation Service (Publisher API) and Event Consumer Service (Consumer API).

5.5.1. API Types

Both services provide dual APIs:

- **gRPC API**: Protocol Buffer-based, high-performance, strongly-typed
- **REST API**: HTTP/JSON-based, easier for testing and web clients

Use gRPC for production microservices and REST for web clients or when easier HTTP tooling is preferred.

5.5.2. Event Aggregation Service (Publisher API)

The Event Aggregation Service accepts events from publishers and routes them to consumers.

5.5.2.1 Base URLs

- **gRPC**: `your-aggregation-service.example.com:50051`
- **REST**: `https://your-aggregation-service.example.com`

5.5.2.2 Authentication

Include a bearer token in all requests:

gRPC:

```
metadata = [('authorization', 'Bearer YOUR_TOKEN')]
```

REST:

```
Authorization: Bearer YOUR_TOKEN
```

5.5.3. Publisher gRPC API

5.5.3.1 Service Definition

```
service EventPublishingService {
  rpc PublishEvent(PublishEventRequest) returns (PublishEventResponse);
  rpc BatchPublishEvents(BatchPublishEventsRequest) returns
↪(BatchPublishEventsResponse);
}
```

5.5.3.2 PublishEvent

Publishes a single event.

Request: `PublishEventRequest`

```
message PublishEventRequest {
  Event event = 1;
}

message Event {
  string event_type = 1;
  google.protobuf.Struct message = 2;
  google.protobuf.Timestamp occurred_at = 3;
  EventResource resource = 4;  // Optional
}

message EventResource {
  string resource_id = 1;
}
```

Response: `PublishEventResponse`

```
message PublishEventResponse {
  PublishingResult result = 1;
}

message PublishingResult {
  Event event = 1;
  bool success = 2;
  string failure_reason = 3;  // Only set if success is false
}
```

Errors:

gRPC Status Code	Meaning
`UNAUTHENTICATED`	Invalid or expired authentication token
`PERMISSION_DENIED`	Not authorized to publish this event type
`INVALID_ARGUMENT`	Invalid event structure
`RESOURCE_EXHAUSTED`	Service temporarily overloaded - retry with backoff
`INTERNAL`	Server error

Example:

```
from nvidia.omniverse.notifications.publisher.v1beta import event_publisher_pb2

# Create event
event = event_publisher_pb2.Event(
    event_type='myapp.user.created',
    message=message_struct,
    occurred_at=timestamp,
    resource=event_publisher_pb2.EventResource(
```

(continues on next page)

(continued from previous page)

```
        resource_id='/users/123'
    )
)

# Publish
request = event_publisher_pb2.PublishEventRequest(event=event)
response = stub.PublishEvent(request, metadata=metadata)

if response.result.success:
    print("Published successfully")
```

5.5.3.3 BatchPublishEvents

Publishes multiple events in parallel.

Request: `BatchPublishEventsRequest`

```
message BatchPublishEventsRequest {
  repeated Event events = 1;
}
```

Response: `BatchPublishEventsResponse`

```
message BatchPublishEventsResponse {
  repeated PublishingResult results = 1;
}
```

Note: Each event in the batch has its own result. Some may succeed while others fail.

Example:

```
# Create multiple events
events = [event1, event2, event3]

# Batch publish
request = event_publisher_pb2.BatchPublishEventsRequest()
for event in events:
    request.events.append(event)
response = stub.BatchPublishEvents(request, metadata=metadata)

# Check individual results
for idx, result in enumerate(response.results):
    if result.success:
        print(f"Event {idx} published")
    else:
        print(f"Event {idx} failed: {result.failure_reason}")
```

5.5.4. Publisher REST API

5.5.4.1 Endpoints

5.5.4.1.1 POST /api/v1beta/events

Publishes a single event.

Request Body:

```
{
  "event": {
    "event_type": "string",
    "message": {},
    "occurred_at": "2024-10-16T14:30:00Z",
    "resource": {
      "resource_id": "/path/to/resource"
    }
  }
}
```

Response (200 OK):

```
{
  "result": {
    "event": { /* echo of event */ },
    "success": true
  }
}
```

Response (200 OK, but publish failed):

```
{
  "result": {
    "event": { /* echo of event */ },
    "success": false,
    "failure_reason": "Unable to connect to message broker"
  }
}
```

Error Responses:

Status Code	Description
400	Invalid request format
401	Unauthenticated
403	Permission denied
422	Validation error
503	Service unavailable - retry

Example:

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "event": {
      "event_type": "myapp.user.created",
      "message": {
        "user_id": "123",
        "username": "john_doe"
      },
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/users/123"
      }
    }
  }'
```

5.5.4.1.2 POST /api/v1beta/events/batch

Publishes multiple events in parallel.

Request Body:

```
{
  "events": [
    {
      "event_type": "string",
      "message": {},
      "occurred_at": "2024-10-16T14:30:00Z",
      "resource": {
        "resource_id": "/path/to/resource"
      }
    }
  ]
}
```

Response (200 OK):

```
{
  "results": [
    {
      "event": { /* event 1 */ },
      "success": true
    },
    {
      "event": { /* event 2 */ },
      "success": false,
      "failure_reason": "Publish failed"
    }
  ]
}
```

Example:

```
curl -X POST https://your-aggregation-service.example.com/api/v1beta/events/batch \
  -H "Authorization: Bearer YOUR_TOKEN" \
```

(continues on next page)

(continued from previous page)

```
  -H "Content-Type: application/json" \
  -d '{
    "events": [
      {
        "event_type": "myapp.file.uploaded",
        "message": {"file": "doc1.pdf"},
        "occurred_at": "2024-10-16T14:30:00Z"
      },
      {
        "event_type": "myapp.file.uploaded",
        "message": {"file": "doc2.pdf"},
        "occurred_at": "2024-10-16T14:30:01Z"
      }
    ]
  }'
```

5.5.5. Event Consumer Service (Consumer API)

The Event Consumer Service allows clients to stream events with filtering.

5.5.5.1 Base URLs

- **gRPC**: `your-consumer-service.example.com:50052`
- **REST**: `https://your-consumer-service.example.com`

5.5.6. Consumer gRPC API

5.5.6.1 Service Definition

```
service EventConsumerService {
  rpc ConsumeNonDurableEvents(stream ConsumeNonDurableEventsRequest)
      returns (stream ConsumeNonDurableEventsResponse);

  // Deprecated: use ConsumeDurableEventsWithAcks instead.
  rpc ConsumeDurableEvents(ConsumeDurableEventsRequest)
      returns (stream ConsumeDurableEventsResponse);

  rpc ConsumeDurableEventsWithAcks(stream ConsumeDurableEventsWithAcksRequest)
      returns (stream ConsumeDurableEventsWithAcksResponse);

  rpc CreateDurableQueue(CreateDurableQueueRequest)
      returns (CreateDurableQueueResponse);

  rpc UpdateDurableQueue(UpdateDurableQueueRequest)
      returns (UpdateDurableQueueResponse);

  rpc DeleteDurableQueue(DeleteDurableQueueRequest)
      returns (DeleteDurableQueueResponse);
```

(continues on next page)

(continued from previous page)

```
  rpc ManageDeadLetteredEvents(stream ManageDeadLetteredEventsRequest)
      returns (stream ManageDeadLetteredEventsResponse);
}
```

5.5.6.2 ConsumeNonDurableEvents

Streams events using a non-durable queue (bidirectional streaming).

Reattaching to the queue it names is what lets you reconnect after a drop and share one queue across parallel clients. When several clients share a queue this way, at most one of them may change its filters, and that client alone can follow the acknowledgement-driven replay recipe described below.

Pass back a token the server issued, verbatim. A token must be 1 to 200 printable ASCII characters — no spaces, no control characters, nothing non-ASCII — and one that is not is rejected with `INVALID_ARGUMENT`, with a message that names the field and the rule and quotes none of the token. The tokens this service issues are random identifiers well inside that rule, so echoing back what you were given always passes it. A token of legal shape that no longer names a live queue is reported as `NOT_FOUND` instead — see below. Omit the field entirely to start a fresh stream.

The stream returns `INTERNAL`, with the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`, when the broker cancels this stream's consumer or kills the channel it lives on — reconnect with the `reconnect_token`, and the queue normally survives with its buffered backlog. That resumes the queue with the bindings it already holds, so a gRPC client that has changed filters mid-stream also re-sends the updates the server did not acknowledge — see the `filter_update_ack` description under **Response** below. `INTERNAL` does not by itself confirm that the queue survived: if the queue no longer exists, the reconnect returns `NOT_FOUND`, and the recovery for a queue that is gone applies. It returns `NOT_FOUND` when the queue is gone: a non-durable queue outlives its consumer only for the deployment's reconnect-grace window (20 seconds by default), after which a `reconnect_token` reconnect returns `NOT_FOUND` again and you should start a fresh stream with `filter_groups` and no token. See *Handle Disconnections Gracefully*.

Request: `ConsumeNonDurableEventsRequest` (stream)

```
message ConsumeNonDurableEventsRequest {
  repeated FilterGroup filter_groups = 1;
  optional string reconnect_token = 2;
  repeated FilterGroup previous_filter_groups = 3;  // Required when updating filters
}

message FilterGroup {
  string event_type = 1;
  repeated ResourceFilter filters = 2;
}

message ResourceFilter {
  enum FilterType {
    FILTER_TYPE_UNSPECIFIED = 0;
    FILTER_TYPE_EQ = 1;
    FILTER_TYPE_STARTS_WITH_LAZY = 2;
    FILTER_TYPE_STARTS_WITH_GREEDY = 3;
  }

  FilterType filter_type = 1;
```

(continues on next page)

(continued from previous page)

```
  string resource_id = 2;
}
```

Response: ConsumeNonDurableEventsResponse (stream)

```
message ConsumeNonDurableEventsResponse {
  repeated Event events = 1;
  string reconnect_token = 2;
  bool supports_filter_update_acks = 3;
  FilterUpdateAck filter_update_ack = 4;
}

message FilterUpdateAck {
  repeated FilterGroup applied_filter_groups = 1;
  bool unchanged = 2;
}

message Event {
  string event_type = 1;
  string principal_identity = 2;
  google.protobuf.Timestamp occurred_at = 3;
  google.protobuf.Timestamp published_at = 4;
  google.protobuf.Struct message = 5;
}
```

`supports_filter_update_acks` is `true` on every response of a stream served by a server that acknowledges the filter sets its requests name. It describes the server serving this stream rather than the individual response, so read it from any response you receive, including the first one as soon as the stream opens. A server that sets it sends one response carrying a `filter_update_ack` for every request it acts on that names a filter set, including a request that required no bind or unbind call because it named the set you had already asked for. A server that does not set it may still act on the filter sets its requests name, but never reports doing so, so you cannot know which of your requests took effect.

`filter_update_ack` is present on a response that acknowledges a request whose filter set the server acted on, and absent on every other response. Receiving it means the server is done with that request: any bind and unbind calls the request led to have returned. It does not report how many calls that was, and the answer may be none. Its two fields:

- `applied_filter_groups` — a verbatim copy of the `filter_groups` of the request being acknowledged. The server holds no filter state and cannot read a queue's bindings back from the broker, so this echoes what you sent rather than reporting what the broker holds.
- `unchanged` — `true` when the acknowledged request's `filter_groups` equalled its `previous_filter_groups` **as you sent them**, so the server issued no bind or unbind call. `false` means the two differed as sent, or that the request named no previous set at all, as a fresh connect does. This is a statement about the request the server received, not about the queue: it does not confirm that the queue's bindings match `applied_filter_groups`, and a `false` value does not promise that any bind or unbind call was issued — a request that re-sends the same groups reordered, or with a group repeated, differs as sent and still changes no bindings.

The first response on the stream carries an acknowledgement when the opening request named a filter set the server acted on. A fresh connect, which names `filter_groups` and no `reconnect_token`, is acknowledged with `unchanged` `false`. A reconnect that names `filter_groups` and `previous_filter_groups` together is a filter update applied before the stream yields anything, and is acknowledged with `unchanged` set according to whether the two sets were equal as you sent them. A reconnect carrying only a `reconnect_token` names no set and gets no acknowledgement,

because the server keeps no filter state and cannot report a set it was not told. A reconnect that names `filter_groups` without `previous_filter_groups` is not a resume either: it is an incomplete filter update and is rejected with `INVALID_ARGUMENT`, so filter groups the server does not apply are never acknowledged as applied. Two kinds of mid-stream request are never acknowledged: one that names neither `filter_groups` nor `previous_filter_groups`, which is ignored, and one that is rejected, which ends the stream with `INVALID_ARGUMENT`. Nor is an update still in flight when the stream ends with a terminal error, as it does when the broker cancels the consumer (see the `INTERNAL` and `NOT_FOUND` cases above): the stream ends and the update is never acknowledged. Acknowledgements are interleaved with event responses in no guaranteed order, but are ordered with respect to one another, in the order the updates were sent. See *Updating Filters Dynamically* for the recipe that uses them to resume a stream whose filters changed.

Example:

```
# Create filter
filter_group = event_consumer_pb2.FilterGroup(
    event_type='myapp.user.created'
)
# No filters added - receive all events of this type

# Request stream
def request_stream():
    request = event_consumer_pb2.ConsumeNonDurableEventsRequest()
    request.filter_groups.append(filter_group)
    yield request

# Consume
responses = stub.ConsumeNonDurableEvents(request_stream(), metadata=metadata)

for response in responses:
    reconnect_token = response.reconnect_token  # Save for reconnection

    for event in response.events:
        print(f"Event: {event.event_type}")
        print(f"Message: {event.message}")
```

5.5.6.3 ConsumeDurableEvents (Deprecated)

> **Deprecated**: Use *`ConsumeDurableEventsWithAcks`* instead. This RPC acknowledges messages automatically after a short delay, which has a window for message loss on disconnect. It will be removed in a future release.

Streams events from a durable queue (server streaming). Messages are automatically acknowledged by the server after a short delay.

This RPC returns `NOT_FOUND` when the queue no longer exists — either the `queue_id` is wrong, or the queue was auto-deleted after going without an attached consumer for the deployment's idle window (24 hours by default); recover by calling `CreateDurableQueue` again with the same `queue_id` and the same `filter_groups`, then resuming. See *Durable Queue Lifetime*.

The stream returns `INTERNAL`, with the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`, when the broker cancels this stream's consumer or kills the channel it lives on. Reconnect and resume consuming; do not recreate the queue. The queue normally survives with its buffered messages, and messages that were in flight unacknowledged are redelivered once you reconnect. `INTERNAL` does not by itself confirm that the queue survived: if the queue no longer exists, the reconnect returns `NOT_FOUND`, and the recovery for a queue that is gone applies. See *Handle Disconnections Gracefully*.

Request: ConsumeDurableEventsRequest

```
message ConsumeDurableEventsRequest {
  string queue_id = 1;
}
```

Field	Type	Description
queue_i	string	Required. The id returned by CreateDurableQueue. It must match ^[A-Za-z0-9._-]{1,200}$; an id that does not is rejected with INVALID_ARGUMENT before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *CreateDurableQueue*.

Response: ConsumeDurableEventsResponse (stream)

```
message ConsumeDurableEventsResponse {
  repeated Event events = 1;
}
```

Example:

```
request = event_consumer_pb2.ConsumeDurableEventsRequest(
    queue_id='saved-queue-id'
)

responses = stub.ConsumeDurableEvents(request, metadata=metadata)

for response in responses:
    for event in response.events:
        process_event(event)
```

5.5.6.4 ConsumeDurableEventsWithAcks

Streams events from a durable queue with explicit client acknowledgment (bidirectional streaming). The client settles each received message by sending its delivery tag back in a subsequent request: in `ack_delivery_tags` to acknowledge it, or in `nack_delivery_tags` to have it delivered again. A nacked message is requeued for another attempt; a message that is neither acknowledged nor nacked is redelivered once the client reconnects. Redelivery is bounded rather than indefinite — the delivery-count budget and the dead-letter queue it ends at are described below.

This RPC returns `NOT_FOUND` when the queue no longer exists — either the `queue_id` is wrong, or the queue was auto-deleted after going without an attached consumer for the deployment's idle window (24 hours by default); recover by calling `CreateDurableQueue` again with the same `queue_id` and the same `filter_groups`, then resuming. See *Durable Queue Lifetime*.

The stream returns `INTERNAL`, with the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`, when the broker cancels this stream's consumer or kills the channel it lives on — an acknowledgement timeout on a slow consumer is the usual cause. Reconnect and resume consuming; do not recreate the queue. The queue normally survives with its buffered messages, and messages that were in flight unacknowledged are redelivered once you reconnect. `INTERNAL` does not by itself confirm that the queue survived: if the queue no longer exists, the reconnect returns `NOT_FOUND`, and the recovery for a queue that is gone applies. See *Handle Disconnections Gracefully*.

Request: `ConsumeDurableEventsWithAcksRequest` (stream)

```
message ConsumeDurableEventsWithAcksRequest {
  string queue_id = 1;                  // Required on first request; ignored after
  repeated int64 ack_delivery_tags = 2;  // Delivery tags to acknowledge
  repeated int64 nack_delivery_tags = 3; // Delivery tags to have delivered again
}
```

The first message in the stream must include `queue_id` to identify the durable queue. It must match `^[A-Za-z0-9._-]{1,200}$`; an id that does not is rejected with `INVALID_ARGUMENT` before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *`CreateDurableQueue`*. Subsequent messages carry `ack_delivery_tags` to acknowledge receipt of previously delivered events, and `nack_delivery_tags` to request redelivery of events the client failed to process. One request may carry both fields; acknowledgments are processed first, so a tag appearing in both is acknowledged. A tag the server does not recognize — already settled, or from an earlier connection — is skipped and does not end the stream.

Settle promptly. An unacknowledged message is redelivered when the client reconnects, and a nacked message is requeued and redelivered **immediately**, possibly on the same stream: the server applies no retry delay, so a client that wants one implements backoff before it nacks.

Every redelivery increments the message's delivery count, whichever way the message was requeued — by a nack, by a disconnect, or by the broker closing the stream's channel. That count is cumulative over the message's whole lifetime and is never reset, so it is a lifetime budget for the message rather than a per-nack one; nacking in a tight loop exhausts it in well under a second. A message whose delivery count passes the deployment's delivery limit is moved to the dead-letter queue rather than redelivered forever — as is one that outlives the queue's configured message TTL or is evicted because the queue reached its configured maximum length. The dead-letter queue is the destination for these poison, expired, and overflowed messages alike. See the *bounded redelivery and dead-letter queue* explanation in the consuming-events guide for the fuller treatment.

Response: `ConsumeDurableEventsWithAcksResponse` (stream)

```
message ConsumeDurableEventsWithAcksResponse {
  repeated Event events = 1;
  int64 delivery_tag = 2;  // Tag to send back to acknowledge or nack this message
}
```

Example:

```
import queue

ack_queue = queue.Queue()

def request_stream(queue_id):
    # First request: identify the durable queue
    yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
        queue_id=queue_id
    )
    # Subsequent requests: send acknowledgments
    while True:
        tags = ack_queue.get()
        if tags is None:
            break
        yield event_consumer_pb2.ConsumeDurableEventsWithAcksRequest(
            ack_delivery_tags=tags
        )
```

(continues on next page)

(continued from previous page)

```
responses = stub.ConsumeDurableEventsWithAcks(
    request_stream('saved-queue-id'), metadata=metadata
)

for response in responses:
    for event in response.events:
        process_event(event)
    # Acknowledge after processing
    ack_queue.put([response.delivery_tag])
```

To hand a message back for another attempt instead, send its `delivery_tag` in `nack_delivery_tags` — after a backoff delay, since redelivery is immediate. The consuming-events guide carries a worked example under *Step 2: Consume from Durable Queue*, and the full retry-budget treatment under *Bounded Redelivery and the Dead-Letter Queue*.

5.5.6.5 CreateDurableQueue

Creates a durable queue for persistent event storage.

This operation has ensure-exists semantics when you supply a `queue_id`: it creates the queue if no queue with that id exists, and is a no-op if a queue with that id already exists. The `created` flag in the response tells you which path the call took.

A `queue_id` you supply must be unique across the whole deployment. Queue ids share a single namespace. If a queue with your id already exists — whoever created it, and whatever `filter_groups` you send — this call attaches to that queue and returns it with its existing filters and `created` set to `false`. Two callers that pick the same id share one queue: each receives the other's events, and either can delete it. Use a UUID, or a namespaced convention such as `<service-name>-<purpose>`; never a bare generic word like `events`.

Per-queue isolation is not enforced. The three durable-queue permissions are deployment-wide rather than per-queue: a principal granted `consume-durable-queues` can consume from any durable queue whose id it knows, one granted `delete-durable-queues` can delete any of them, and `create-durable-queues` authorizes both creating a queue and updating the filters of any existing one. Treat a `queue_id` as configuration, not as a secret that protects a queue, and ask your operator to grant these permissions to the service accounts that consume events rather than to broad user groups.

The queue exists until you delete it with `DeleteDurableQueue`, or until it has gone without an attached consumer for the deployment's idle window (24 hours by default), at which point it is deleted along with any messages it still holds. Only an attached consumer keeps a durable queue alive — publishing to it and re-declaring it do not. Consuming from a queue deleted this way returns `NOT_FOUND`; recover by calling this RPC again with the same `queue_id` and the same `filter_groups`, since the queue's subscriptions are rebuilt from the filters. See *Durable Queue Lifetime*.

Request: `CreateDurableQueueRequest`

```
message CreateDurableQueueRequest {
  repeated FilterGroup filter_groups = 1;
  string queue_id = 2;
}
```

Field	Type	Description
filter_groups	repeated FilterGroup	Filters to apply to events. If omitted or empty, all events for an event type are added to the queue.
queue_id	string	Optional, client-specified queue identifier. An empty string means not supplied — the server generates one. When supplied, it must match `^[A-Za-z0-9._-]{1,200}$` (an id that fails this rule is rejected with `INVALID_ARGUMENT`) and it must be unique across the whole deployment — an id already in use attaches to that existing queue rather than creating one.

Response: `CreateDurableQueueResponse`

```
message CreateDurableQueueResponse {
  string queue_id = 1;  // Save this, together with your filter_groups!
  bool created = 2;
}
```

Field	Type	Description
queue_id	string	The id of the durable queue. Persist this **together with the `filter_groups` you created it with** and send the id on all `ConsumeDurableEvents` calls. Both are needed to recreate the queue if it is auto-deleted after going unconsumed for the idle window, since its subscriptions are rebuilt from the filters — see *Durable Queue Lifetime*.
created	bool	`true` when this call created a new queue, `false` when an existing queue with that id was found and returned unchanged (a no-op).

Example:

```
filter_group = event_consumer_pb2.FilterGroup(
    event_type='myapp.file.uploaded'
)
resource_filter = event_consumer_pb2.ResourceFilter(
    filter_type=event_consumer_pb2.ResourceFilter.FILTER_TYPE_STARTS_WITH_GREEDY,
    resource_id='/uploads/'
)
filter_group.filters.append(resource_filter)

request = event_consumer_pb2.CreateDurableQueueRequest()
request.filter_groups.append(filter_group)

# Optional: supply your own queue_id for ensure-exists semantics. Omit it
# (leave it empty) to have the server generate one. A supplied id must be
# unique across the deployment, so namespace it as <service-name>-<purpose>.
request.queue_id = 'myapp-uploads-queue'

response = stub.CreateDurableQueue(request, metadata=metadata)
queue_id = response.queue_id

if response.created:
```

(continues on next page)

(continued from previous page)

```
    print(f'Created new durable queue {queue_id}')
else:
    print(f'Reusing existing durable queue {queue_id}')

# IMPORTANT: Save queue_id AND the filter_groups it was created with to
# configuration — both are needed to recreate the queue if it is auto-deleted
# after going unconsumed for the idle window. See "Durable Queue Lifetime"
# (consuming-events.md#durable-queue-lifetime).
```

5.5.6.6 UpdateDurableQueue

Updates filters on an existing durable queue.

This RPC is authorized by the `create-durable-queues` permission — the same one that authorizes creating a queue. There is no separate update permission, so a principal that can create durable queues can re-filter any existing one.

Request: `UpdateDurableQueueRequest`

```
message UpdateDurableQueueRequest {
  string queue_id = 1;
  repeated FilterGroup current_filter_groups = 2;
  repeated FilterGroup new_filter_groups = 3;
}
```

Field	Type	Description
`queue_id`	`string`	Required. The id of the durable queue to update. It must match `^[A-Za-z0-9._-]{1,200}$`; an id that does not is rejected with `INVALID_ARGUMENT` before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *CreateDurableQueue*.
`current_filter_gr`	`repeated FilterGroup`	The filters currently in use on the queue.
`new_filter_gro`	`repeated FilterGroup`	The filters to use instead.

Response: `UpdateDurableQueueResponse`

```
message UpdateDurableQueueResponse {}
```

5.5.6.7 DeleteDurableQueue

Deletes a durable queue.

Request: `DeleteDurableQueueRequest`

```
message DeleteDurableQueueRequest {
  string queue_id = 1;
}
```

Field	Type	Description
queue_i	string	Required. The id of the durable queue to delete. It must match `^[A-Za-z0-9._-]{1,200}$`; an id that does not is rejected with `INVALID_ARGUMENT` before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *CreateDurableQueue*.

Response: `DeleteDurableQueueResponse`

```
message DeleteDurableQueueResponse {}
```

Example:

```
request = event_consumer_pb2.DeleteDurableQueueRequest(
    queue_id=queue_id
)

stub.DeleteDurableQueue(request, metadata=metadata)
```

5.5.6.8 ManageDeadLetteredEvents

Operator-facing, bidirectional streaming RPC for inspecting and triaging dead-lettered events across the deployment. The server streams dead-lettered messages with triage metadata; the client replies with per-message actions (`ACTION_REPLAY`, `ACTION_DISCARD`, `ACTION_LEAVE`) keyed by a session-scoped delivery tag; the server streams back a correlated outcome report for each action.

gRPC-only: There is no REST endpoint for this RPC.

Required permission: `event-consumer-service:inspect-and-replay-all-dlq`. A caller lacking it is denied with `PERMISSION_DENIED`.

The stream returns `INTERNAL`, with the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.`, when the broker cancels this session's consumer or kills the channel it lives on. Reconnect to resume triage: the dead-letter queue and everything in it normally survive, and dead-lettered messages that were in flight unacknowledged are redelivered once you reconnect. `NOT_FOUND`, on the stream or on the reconnect, means the deployment's dead-letter queue is missing on the broker, which is an operator condition rather than something the caller can recover from.

This no-store bidirectional RPC is the permanent baseline DLQ-management surface; it requires no datastore beyond the message broker. It is the forward-compatible floor: future additive unary store-backed RPCs (list/filter/replay-by-id/stats) may be added later without changing this RPC. See *Managing the Dead-Letter Queue (Operators)* in the consuming-events guide.

Request: `ManageDeadLetteredEventsRequest` (stream)

```
message ManageDeadLetteredEventsRequest {
  enum Action {
    ACTION_UNSPECIFIED = 0;
    ACTION_REPLAY = 1;   // Republish to origin queue (confirm-gated, no-loss)
    ACTION_DISCARD = 2;  // Ack and drop from the dead-letter queue
    ACTION_LEAVE = 3;    // Leave in place for later triage
  }

  message ActionEntry {
    int64 delivery_tag = 1;  // Session-scoped tag of the message to act on
    Action action = 2;       // Action to take on that message
  }

  repeated ActionEntry actions = 1;  // Empty on the first (opening) message
}
```

The first message opens the stream and carries no actions (the dead-letter queue is global, so there is no `queue_id`). Subsequent messages carry one or more `ActionEntry` items, each pairing a session-scoped `delivery_tag` with the `Action` to take, so multiple dead-lettered messages can be actioned in a single request.

Request fields:

Field	Type	Description
`actions`	`repeated ActionEntry`	Per-delivery-tag triage actions. Empty on the opening message.
`ActionEntry.delivery_tag`	`int64`	Session-scoped tag of the dead-lettered message, as received in a `dead_lettered_event`. Valid only within the live stream; not persisted or reused across reconnects.
`ActionEntry.action`	`Action`	`ACTION_REPLAY`, `ACTION_DISCARD`, or `ACTION_LEAVE`.

Response: `ManageDeadLetteredEventsResponse` (stream)

Each response carries exactly one of two arms via a `oneof payload`: a `dead_lettered_event` (a browse item) or an `action_outcome` (a report correlated to a requested action by `delivery_tag`).

```
message ManageDeadLetteredEventsResponse {
  message DeadLetteredEvent {
    enum Reason {
      REASON_UNSPECIFIED = 0;
      REASON_DELIVERY_LIMIT = 1;  // Exceeded its redelivery limit
      REASON_EXPIRED = 2;         // Outlived the queue's configured message TTL
      REASON_MAXLEN = 3;          // Evicted when queue reached its maximum length
    }

    Reason reason = 1;
    string origin_queue = 2;                          // Origin durable queue name
    int64 delivery_count = 3;                         // Broker delivery attempts
    google.protobuf.Timestamp dead_lettered_at = 4;  // When it was dead-lettered
    Event event = 5;                                  // event_type, principal_identity,
↪message
```

(continues on next page)

(continued from previous page)

```
    string resource_id = 6;                          // Resource id of the event
    int64 delivery_tag = 7;                          // Session-scoped tag to act on
  }

  message ActionOutcome {
    enum Outcome {
      OUTCOME_UNSPECIFIED = 0;
      OUTCOME_REPLAYED = 1;              // Republished to origin queue; DLQ copy
↪removed
      OUTCOME_DISCARDED = 2;             // Acked and dropped from the dead-letter queue
      OUTCOME_LEFT = 3;                  // Left in place for later triage
      OUTCOME_ORIGIN_QUEUE_DELETED = 4;  // Origin queue gone; DLQ copy left in place
↪(no-loss)
      OUTCOME_REPLAY_FAILED = 5;         // Replay unconfirmed; DLQ copy left in place
↪(no-loss)
    }

    int64 delivery_tag = 1;  // Matches the tag the client sent
    Outcome outcome = 2;
    string detail = 3;       // Human-readable context (e.g. missing queue name)
  }

  oneof payload {
    DeadLetteredEvent dead_lettered_event = 1;
    ActionOutcome action_outcome = 2;
  }
}
```

`DeadLetteredEvent` (browse item) fields:

Field	Type	Description
`reason`	`Reason`	Why the message was dead-lettered: `REASON_DELIVERY_LIMIT`, `REASON_EXPIRED`, or `REASON_MAXLEN`.
`origin_queue`	`string`	Name of the durable queue the message was dead-lettered from.
`delivery_count`	`int64`	Broker delivery attempts before dead-lettering.
`dead_lettere(`	`Timestamp`	When the message was dead-lettered.
`event`	`Event`	The dead-lettered event (`event_type`, `principal_identity`, `message`).
`resource_id`	`string`	Resource id associated with the event.
`delivery_tag`	`int64`	Session-scoped tag the client sends back to act on the message. Livestream-only.

`ActionOutcome` (report) fields:

Field	Type	Description
`delivery_tag`	`int64`	Session-scoped tag identifying the message the outcome pertains to.
`outcome`	`Outcome`	`OUTCOME_REPLAYED`, `OUTCOME_DISCARDED`, `OUTCOME_LEFT`, `OUTCOME_ORIGIN_QUEUE_DELETED`, or `OUTCOME_REPLAY_FAILED`.
`detail`	`string`	Human-readable context (e.g. the missing origin queue name).

Each action the operator sends produces exactly one correlated `action_outcome`, keyed by `delivery_tag`, so the operator can observe the result of every action. `ACTION_REPLAY` is confirm-gated and no-loss: the dead-letter copy is removed only after the publish to the origin queue is confirmed. A `ACTION_REPLAY` whose origin queue has been deleted does **not** lose the message — the server leaves the dead-letter copy in place (ACTION_LEAVE semantics) and reports an `OUTCOME_ORIGIN_QUEUE_DELETED` outcome to the operator rather than silently handling it; re-create the origin queue to obtain a viable replay target. A replay that cannot be confirmed reports `OUTCOME_REPLAY_FAILED` and likewise leaves the copy in place.

Scope: `ACTION_REPLAY` returns a message to the exact origin queue it dead-lettered from, taken from the message's own broker metadata — an operator cannot redirect a replay to another queue. The permission is coarse and grants deployment-wide dead-letter visibility and action by design; re-consume of a replayed message still runs the existing per-event authorization check.

5.5.7. Consumer REST API

5.5.7.1 Endpoints

5.5.7.1.1 GET /api/v1beta/events/stream

Streams events using a non-durable queue (Server-Sent Events).

Query Parameters:

Parameter	Type	Required	Description
`filter_groups`	string (JSON)	No	URL-encoded JSON array of FilterGroup objects
`previous_filter_grou`	string (JSON)	No	URL-encoded JSON array of the previous FilterGroup objects (required when updating filters)
`reconnect_token`	string	No	**Deprecated — send the token in the `Last-Event-ID` header instead** (see below). Still accepted, and carries the token the same way. Token for reconnecting to the same queue after a disconnect, or for consuming from the same queue in parallel from multiple clients. Must be 1 to 200 printable ASCII characters.

Headers:

Header	Required	Description
`Last-Event-`	No	The recommended way to send a reconnect token back. Carries exactly the value the `reconnect_token` query parameter would, supports every operation it does — a filter update included — and takes the same 1-to-200-printable-ASCII rule. A browser's built-in SSE reconnect sends this header automatically.

Reconnecting and updating filters: sending `previous_filter_groups` alongside `filter_groups` is what makes a request a filter update, whichever carrier brought the token. A request that carries the token alone resumes the stream on the filters it already has. A browser's automatic reconnect replays the URL the stream was opened on and sends the token in the `Last-Event-ID` header, so when that URL carried only the initial `filter_groups`, the header path resumes as-is.

The token identifies the queue. It is not a secret and grants no access on its own: presenting it requires a valid bearer token, and every delivered event is authorized against the presenting principal's own permissions. Consumers attached to one queue split its events between them, so keeping the token within the application that received it is what ensures only that application's own authorized consumers share those events.

FilterGroup Structure:

```
[
  {
    "event_type": "string",
    "filters": [
      {
        "filter_type": "starts_with_greedy",
        "resource_id": "/path/"
      }
    ]
  }
]
```

Filter Types:

- `eq`: Exact match
- `starts_with_lazy`: Shallow prefix match
- `starts_with_greedy`: Deep prefix match
- `unspecified`: Defaults to `eq`

Response (text/event-stream):

```
event: connected
id: my-queue-id
data: {"message":"Connection established","reconnect_token":"my-queue-id"}

event: event
data: {"event_type":"myapp.user.created","principal_identity":"user@example.com",
↪"occurred_at":"2024-10-16T14:30:00Z","published_at":"2024-10-16T14:30:01Z","message
↪":{"user_id":"123"}}
```

The initial `connected` event includes the `reconnect_token` (which is the queue ID). Save this token for reconnection; it also lets multiple clients consume from the same queue in parallel, splitting its

events between them.

Errors on an open stream: the HTTP response status is `200 OK` as soon as the stream opens, so an error that ends the stream is not an HTTP status — it is delivered as an SSE event named `error` whose `data` carries a `message` and the `status_code` it would have had, after which the stream closes. Handle the `error` event explicitly; a client that watches only for `event` receives no further events and no visible failure. The stream ends this way with `status_code 500` and the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.` when the broker takes this stream's consumer away — reconnect with your `reconnect_token`, and the queue normally survives with its buffered backlog — and with `status_code 404` when the queue itself is gone, in which case start a fresh stream from your `filter_groups` with no `reconnect_token`. The `500` does not by itself confirm that the queue survived: if the queue no longer exists, the reconnect returns `404`, and the recovery for a queue that is gone applies. See *Handle Disconnections Gracefully*.

Example:

```
# Simple consumption
curl -N https://your-consumer-service.example.com/api/v1beta/events/stream \
  -H "Authorization: Bearer YOUR_TOKEN"

# With filtering
FILTERS='[{"event_type":"myapp.user.created","filters":[]}]'
curl -N https://your-consumer-service.example.com/api/v1beta/events/stream \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -G --data-urlencode "filter_groups=$FILTERS"
```

5.5.7.1.2 GET /api/v1beta/events/stream-durable

Streams events from a durable queue (Server-Sent Events). Messages are acknowledged to the broker automatically by the server after a short delay. Clients do not need to acknowledge individual messages.

If your application requires explicit client-side acknowledgment of each message (e.g. to guarantee at-least-once processing), or needs to negatively acknowledge a message it failed to process so the broker redelivers it, use the gRPC *ConsumeDurableEventsWithAcks* RPC instead. Its request carries both `ack_delivery_tags` and `nack_delivery_tags`. That RPC uses a bidirectional stream, which keeps the acknowledgment on the same connection and works correctly across multi-pod deployments — a delivery tag is scoped to the broker channel that issued it, so a separate REST call could not settle it reliably.

This endpoint returns `404` when the queue no longer exists — either the `queue_id` is wrong, or the queue was auto-deleted after going without an attached consumer for the deployment's idle window (24 hours by default); recover by creating the queue again with the same `queue_id` and the same `filter_groups`, then resuming. See *Durable Queue Lifetime*.

Errors on an open stream: once the stream has opened its HTTP response status is `200 OK`, so a status that arises after that point — including the `404` above, when the queue is deleted or reaped out from under a running stream — is not an HTTP status. It is delivered as an SSE event named `error` whose `data` carries a `message` and the `status_code` it would have had, after which the stream closes. Handle the `error` event explicitly; a client that watches only for `event` receives no further events and no visible failure. The stream also ends this way with `status_code 500` and the message `Internal Server Error: The broker cancelled this stream's consumer. Reconnect to resume consuming.` when the broker takes this stream's consumer away — reconnect with the same `queue_id`, and do not recreate the queue; the queue normally survives with everything buffered in it. The `500` does not by itself confirm that the queue survived: if the queue no longer exists, the

reconnect returns 404, and the recovery for a queue that is gone applies. See *Handle Disconnections Gracefully*.

Query Parameters:

Parameter	Type	Required	Description
queue_id	string	Yes	ID of the durable queue. It must match ^[A-Za-z0-9._-]{1,200}$; an id that does not is rejected with 422 Unprocessable Entity before the stream opens. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *POST /api/v1beta/queues/durable*.

Response (text/event-stream):

```
event: connected
id: my-queue-id
data: {"message":"Connection established","reconnect_token":"my-queue-id"}

event: event
data: {"event_type":"myapp.user.created",...}

event: event
data: {"event_type":"myapp.user.updated",...}
```

Example:

```
curl -N https://your-consumer-service.example.com/api/v1beta/events/stream-durable?
↪queue_id=my-queue-id \
  -H "Authorization: Bearer YOUR_TOKEN"
```

5.5.7.1.3 POST /api/v1beta/queues/durable

Creates a durable queue.

This endpoint has ensure-exists semantics when you supply queue_id: it creates the queue if no queue with that id exists, and is a no-op if a queue with that id already exists. The created field in the response tells you which path the call took. An invalid queue_id (one that fails the ^[A-Za-z0-9._-]{1,200}$ validation rule) is rejected with 422 Unprocessable Entity.

A queue_id you supply must be unique across the whole deployment. Queue ids share a single namespace. If a queue with your id already exists — whoever created it, and whatever filter_groups you send — this endpoint attaches to that queue and returns it with its existing filters and created set to false. Two callers that pick the same id share one queue: each receives the other's events, and either can delete it. Use a UUID, or a namespaced convention such as <service-name>-<purpose>; never a bare generic word like events.

Per-queue isolation is not enforced. The three durable-queue permissions are deployment-wide rather than per-queue: a principal granted consume-durable-queues can consume from any durable queue whose id it knows, one granted delete-durable-queues can delete any of them, and create-durable-queues authorizes both creating a queue and updating the filters of any existing one. Treat a queue_id as configuration, not as a secret that protects a queue.

Request Body:

```
{
  "filter_groups": [
    {
      "event_type": "string",
      "filters": [
        {
          "filter_type": "starts_with_greedy",
          "resource_id": "/path/"
        }
      ]
    }
  ],
  "queue_id": "myapp-uploads-queue"
}
```

`queue_id` is optional and client-specified. Omit it or send an empty string to have the server generate one. When supplied, it must match `^[A-Za-z0-9._-]{1,200}$` and must be unique across the deployment, as described above.

The HTTP status code signals which path the request took:

- `201 Created` — the request created a new durable queue (`created` is `true`). This is the case when you supply a `queue_id` that does not yet exist.
- `200 OK` — the request resolved to a queue that already existed (`created` is `false`), or you omitted `queue_id` and the server generated one (`created` is `true`).

Response (`201 Created`, created-new path):

```
{
  "queue_id": "durable-queue-abc123",
  "created": true
}
```

`created` is `true` when this call created a new queue and `false` when an existing queue with the supplied `queue_id` was found and returned unchanged (a no-op). Branch on the status code to tell the two apart.

Example:

```
curl -X POST https://your-consumer-service.example.com/api/v1beta/queues/durable \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "filter_groups": [
      {
        "event_type": "myapp.file.uploaded",
        "filters": [
          {
            "filter_type": "starts_with_greedy",
            "resource_id": "/uploads/"
          }
        ]
      }
    ],
    "queue_id": "myapp-uploads-queue"
  }'
```

5.5.7.1.4 PATCH /api/v1beta/queues/durable

Updates filters on a durable queue.

This endpoint is authorized by the `create-durable-queues` permission — the same one that authorizes creating a queue. There is no separate update permission.

Request Body:

```
{
  "queue_id": "string",
  "current_filter_groups": [ /* array of FilterGroup */ ],
  "new_filter_groups": [ /* array of FilterGroup */ ]
}
```

`queue_id` must match `^[A-Za-z0-9._-]{1,200}$`; an id that does not is rejected with `422 Unprocessable Entity` before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *POST /api/v1beta/queues/durable*.

Response (200 OK):

```
{
  "success": true
}
```

5.5.7.1.5 DELETE /api/v1beta/queues/durable

Deletes a durable queue.

Query Parameters:

Parameter	Type	Required	Description
queue_i	string	Yes	ID of the queue to delete. It must match `^[A-Za-z0-9._-]{1,200}$`; an id that does not is rejected with `422 Unprocessable Entity` before the broker is contacted. Queue ids share one deployment-wide namespace, so this must be the id of a queue that was created by (or for) your client; see *POST /api/v1beta/queues/durable*.

Response (200 OK):

```
{
  "success": true
}
```

Example:

```
curl -X DELETE "https://your-consumer-service.example.com/api/v1beta/queues/durable?
↪queue_id=my-queue-id" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

5.5.8. Error Handling

5.5.8.1 Common HTTP Status Codes

Code	Meaning	Action
200	Success	Process response
400	Bad Request	Check request format
401	Unauthorized	Refresh authentication token
403	Forbidden	Check permissions
404	Not Found	Check queue_id or endpoint
422	Validation Error	Check request validation
500	Internal Server Error	Retry after delay, contact support. On an SSE consume stream this status arrives inside an `error` event rather than as the response status (the response is already `200 OK`), and the stream then closes; `The broker cancelled this stream's consumer.` means reconnect and resume instead — see *Handle Disconnections Gracefully*
503	Service Unavailable	Retry with exponential backoff

5.5.8.2 Common gRPC Status Codes

Code	Meaning	Action
OK	Success	Process response
INVALID_ARGUM	Bad request	Check request format
UNAUTHENTICATED	Auth failed	Refresh token
PERMISSION_DENIED	No permission	Check permissions
NOT_FOUND	Resource not found	Check identifiers
RESOURCE_EXHA	Service overloaded	Retry with backoff
UNAVAILABLE	Connection lost (e.g. load balancer idle timeout)	Reconnect automatically
INTERNAL	Server error	Retry after delay. On a consume stream, `The broker cancelled this stream's consumer.` means reconnect and resume instead — see *Handle Disconnections Gracefully*

5.5.9. Protocol Buffer Files

The complete protocol buffer definitions are available in the released archive or protobuf registry.

Publisher Proto: `nvidia/omniverse/notifications/publisher/v1beta/event_publisher.proto`

Consumer Proto: `nvidia/omniverse/notifications/consumer/v1beta/event_consumer.proto`

5.5.10. OpenAPI Specifications

Full OpenAPI specifications are available in the released archive

5.5.11. Rate Limits and Quotas

Currently, there are no enforced rate limits at the application level. However:

- The service may return `RESOURCE_EXHAUSTED` (gRPC) or `503` (HTTP) if temporarily overloaded
- Implement exponential backoff when you receive these errors
- For high-volume publishing, use batch publishing for better performance

5.5.12. Versioning

The current API version is **v1beta**.

- APIs are under the `/api/v1beta/` path (REST) or `v1beta` package (gRPC)
- Beta APIs may change, but we will maintain backward compatibility where possible
- When the API reaches GA, it will be versioned as v1

5.5.13. Service Health

Both services expose health check endpoints:

gRPC: Standard gRPC health checking protocol

REST:

- GET `/health` - Service health status
- Returns 200 if healthy

5.5.14. Metrics and Monitoring

Both services expose OpenTelemetry metrics and traces when telemetry is enabled via configuration.

5.5.15. Next Steps

- Try the *Getting Started Guide* for hands-on examples
- Learn about *Publishing Events*
- Learn about *Consuming Events*
- Understand *Permissions & Authorization*

Chapter 6. Quick Links

- *Getting Started Guide* - Set up and run your first example
- *Publishing Events* - Learn how to publish events (REST and gRPC)
- *Consuming Events* - Learn how to consume events (REST and gRPC)
- *Permissions & Authorization* - Understanding access control
- *API Reference* - Detailed API documentation

Chapter 7. Support and Contributing

For questions, issues, or contributions, please contact the Omniverse Notifications Service team.

Copyright

©2024-2026, NVIDIA Corporation